

.NET Technologies

SIUSCS46

By Shivani Deopa

The .NET Framework

- .NET Core is the latest general purpose development platform maintained by Microsoft. It works across different platforms and has been redesigned in a way that makes .NET fast, flexible and modern.
 - This happens to be one of the major contributions by Microsoft. Developers can now build Android, iOS, Linux, Mac, and Windows applications with .NET, all in Open Source.
 - Characteristics of .NET Core
 - Open source
1. .NET Core is an open source implementation, using MIT and Apache 2 licenses.
 2. .NET Core is a .NET Foundation project and is available on GitHub.
 3. As an open source project, it promotes a more transparent development process and promotes an active and engaged community.

- Cross-platform
 1. Application implemented in .NET Core can be run and its code can be reused regardless of your platform target.
 2. It currently supports three main operating systems (OS)
 3. Windows
 4. Linux
 5. MacOS
 6. The supported Operating Systems (OS), CPUs and application scenarios will grow over time, provided by Microsoft, other companies, and individuals.
- Flexible deployment
 1. There can be two types of deployments for .NET Core applications –
 2. Framework-dependent deployment
 3. Self-contained deployment
 4. With framework-dependent deployment, your app depends on a system-wide version of .NET Core on which your app and third-party dependencies are installed.
 5. With self-contained deployment, the .NET Core version used to build your application is also deployed along with your app and third-party dependencies and can run side-by-side with other versions.

- Command-line tools

1. All product scenarios can be exercised at the command-line.

- Compatible

1. .NET Core is compatible with .NET Framework, Xamarin and Mono, via the .NET Standard Library

- Modular

1. .NET Core is released through NuGet in smaller assembly packages.

2. .NET Framework is one large assembly that contains most of the core functionalities.

3. .NET Core is made available as smaller feature-centric packages.

4. This modular approach enables the developers to optimize their app by including just those NuGet packages which they need in their app.

5. The benefits of a smaller app surface area include tighter security, reduced servicing, improved performance, and decreased costs in a pay-for-what-you-use model.

.NET Languages

- .NET is a software framework which is designed and developed by Microsoft. The first version of the .Net framework was 1.0 which came in the year 2002.
- In easy words, it is a virtual machine for compiling and executing programs written in different languages like C#, VB.Net etc.
- It is used to develop Form-based applications, Web-based applications, and Web services.
- There is a variety of programming languages available on the .Net platform, VB.Net and C# being the most common ones. It is used to build applications for Windows, phone, web, etc. It provides a lot of functionalities and also supports industry standards.
- .NET Framework supports more than 60 programming languages in which 11 programming languages are designed and developed by Microsoft.
- The remaining Non-Microsoft Languages which are supported by .NET Framework but not designed and developed by Microsoft.

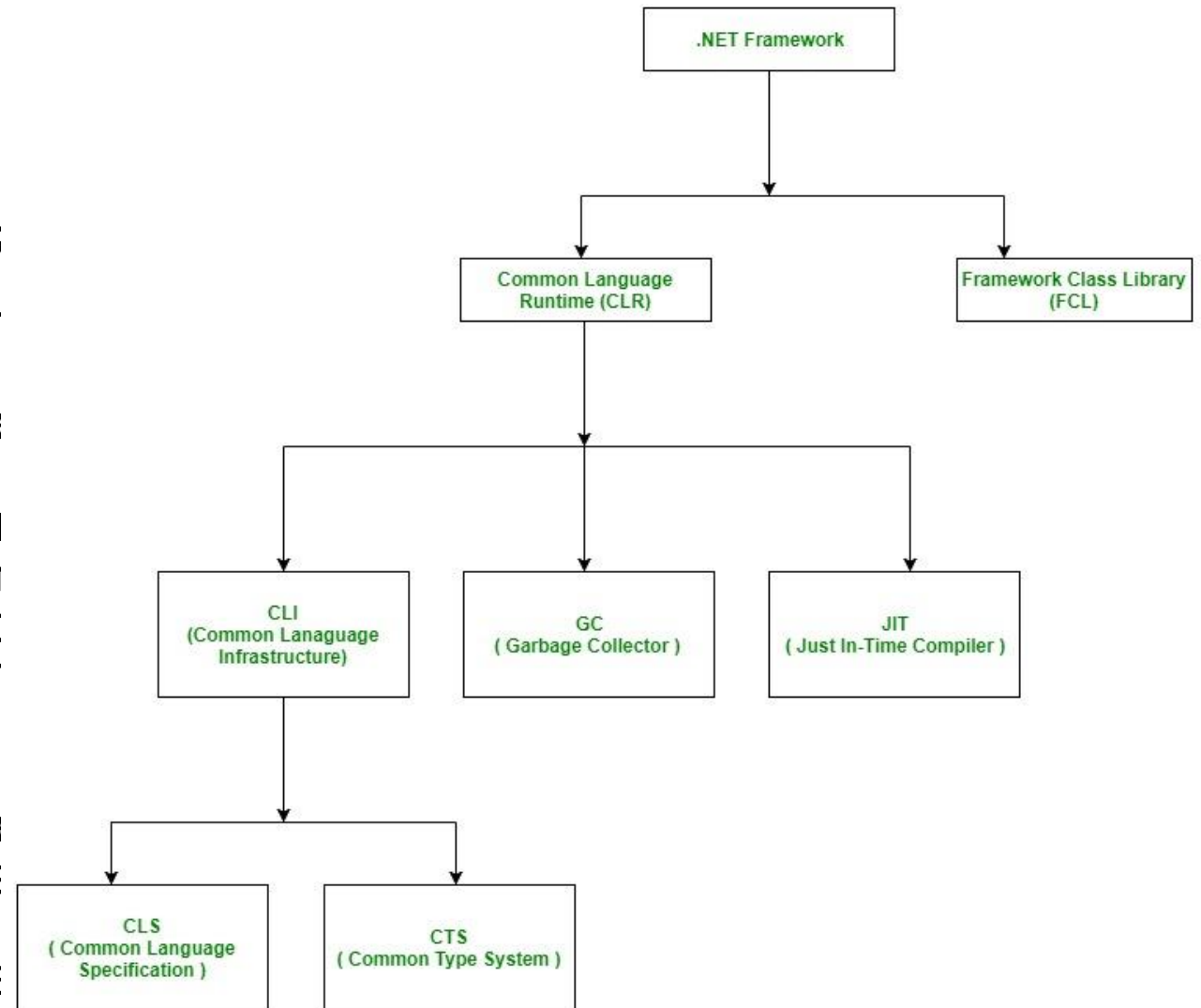
11 Programming Languages which are designed and developed by Microsoft are:

- C#.NET
- VB.NET
- C++.NET
- J#.NET
- F#.NET
- JSCRIPT.NET
- WINDOWS POWERSHELL
- IRON RUBY
- IRON PYTHON
- C OMEGA
- ASML(Abtract State Machine Language)

Main Components

- Common Language Runtime(CLR)
CLR is the basic and Virtual Machine run-time environment in the .NET making the development process of remoting, thread management, type etc.. Basically, it is responsible for the execution of any .NET program regardless of any .NET programming code, as code that targets the runtime doesn't target to runtime is known as metadata.
- Framework Class Library(FCL):

It is the collection of reusable, object-oriented classes that can be integrated with CLR. Also called as the standard library in C/C++ and packages in the java. The installation of CLR and FCL into the system is done by the .NET Framework Setup.



C# Language

- C# is a general-purpose, modern and object-oriented programming language pronounced as “C sharp”.
- It was developed by Microsoft led by Anders Hejlsberg and his team within the .Net initiative and was approved by the European Computer Manufacturers Association (ECMA) and International Standards Organization (ISO).
- C# is among the languages for Common Language Infrastructure and the current version of C# is version 7.2. C# is a lot similar to Java syntactically and is easy for the users who have knowledge of C, C++ or Java.

Why C#?

- C# has many other reasons for being popular and in demand. Few of the reasons are mentioned below:
 1. Easy to start: C# is a high-level language so it is closer to other popular programming languages like C, C++, and Java and thus becomes easy to learn for anyone.
 2. Widely used for developing Desktop and Web Application: C# is widely used for developing web applications and Desktop applications. It is one of the most popular languages that is used in professional desktop. If anyone wants to create Microsoft apps, C# is their first choice.
 3. Community: The larger the community the better it is as new tools and software will be developing to make it better. C# has a large community so the developments are done to make it exist in the system and not become extinct.
 4. Game Development: C# is widely used in game development and will continue to dominate. C# integrates with Microsoft and thus has a large target audience. The C# features such as Automatic Garbage Collection, interfaces, object-oriented, etc. make C# a popular game developing language.

C# Syntax

```
using System;
```

```
namespace HelloWorld
```

```
{
```

```
    class Program
```

```
    {
```

```
        static void Main(string[] args)
```

```
        {
```

```
            Console.WriteLine("Hello World!");
```

```
        }
```

```
    }
```

```
}
```

Comments

- Single-line comments start with two forward slashes (//). Any text between // and the end of the line is ignored by C#

Ex:

```
// This is a comment
```

```
Console.WriteLine("Hello World!");
```

- Multi-line comments start with /* and ends with */. Any text between /* and */ will be ignored by C#.

Ex:

```
/* The code below will print the words Hello World  
to the screen, and it is amazing */
```

```
Console.WriteLine("Hello World!");
```

Variables and Data Types

- Variables are containers for storing data values.
- In C#, there are different types of variables (defined with different keywords), for example:
 1. int - stores integers (whole numbers), without decimals, such as 123 or -123
 2. double - stores floating point numbers, with decimals, such as 19.99 or -19.99
 3. char - stores single characters, such as 'a' or 'B'. Char values are surrounded by single quotes
 4. string - stores text, such as "Hello World". String values are surrounded by double quotes
 5. bool - stores values with two states: true or false

Example:

```
int myNum = 5;           // Integer (whole number)
double myDoubleNum = 5.99D; // Floating point number
char myLetter = 'D';      // Character
bool myBool = true;       // Boolean
string myText = "Hello";  // String
```

Variable Operations

- In C#, there are two types of casting:
- **Implicit Casting (automatically)** - converting a smaller type to a larger type size
char -> int -> long -> float -> double

Example:

```
int myInt = 9;  
double myDouble = myInt;    // Automatic casting: int to double  
Console.WriteLine(myInt);    // Outputs 9  
Console.WriteLine(myDouble); // Outputs 9
```

- **Explicit Casting (manually)** - converting a larger type to a smaller size type
double -> float -> long -> int -> char

Example:

```
double myDouble = 9.78;  
int myInt = (int) myDouble; // Manual casting: double to int  
Console.WriteLine(myDouble); // Outputs 9.78  
Console.WriteLine(myInt);    // Outputs 9
```

- It is also possible to convert data types explicitly by using built-in methods, such as

1. Convert.ToBoolean,
2. Convert.ToDouble,
3. Convert.ToString,
4. Convert.ToInt32 (int) and
5. Convert.ToInt64 (long)

Example:

```
int myInt = 10;
```

```
double myDouble = 5.25;
```

```
bool myBool = true;
```

```
Console.WriteLine(Convert.ToString(myInt)); // convert int to string
```

```
Console.WriteLine(Convert.ToDouble(myInt)); // convert int to double
```

```
Console.WriteLine(Convert.ToInt32(myDouble)); // convert double to int
```

```
Console.WriteLine(Convert.ToString(myBool)); // convert bool to string
```

Get User Input

- You have already learned that `Console.WriteLine()` is used to output (print) values. Now we will use `Console.ReadLine()` to get user input.
- In the following example, the user can input his or hers username, which is stored in the variable `userName`. Then we print the value of `userName`:

Example:

```
Console.WriteLine("Enter username:");  
string userName = Console.ReadLine();  
Console.WriteLine("Username is: " + userName);
```

- The `Console.ReadLine()` method returns a string. Therefore, you cannot get information from another data type, such as `int`. The following program will cause an error:

Example:

```
Console.WriteLine("Enter your age:");  
int age = Convert.ToInt32(Console.ReadLine());  
Console.WriteLine("Your age is: " + age);
```

Method

- A method is a group of statements that together perform a task. Every C# program has at least one class with a method named Main.
- To use a method, you need to –
 1. Define the method
 2. Call the method
- Defining Methods in C#

When you define a method, you basically declare the elements of its structure. The syntax for defining a method in C# is as follows –

```
<Access Specifier> <Return Type> <Method Name>(Parameter List) {  
    Method Body  
}
```


- Following are the various elements of a method –
- Access Specifier – This determines the visibility of a variable or a method from another class.
- Return type – A method may return a value. The return type is the data type of the value the method returns. If the method is not returning any values, then the return type is void.
- Method name – Method name is a unique identifier and it is case sensitive. It cannot be same as any other identifier declared in the class.
- Parameter list – Enclosed between parentheses, the parameters are used to pass and receive data from a method. The parameter list refers to the type, order, and number of the parameters of a method. Parameters are optional; that is, a method may contain no parameters.
- Method body – This contains the set of instructions needed to complete the required activity.

Example:

```
class NumberManipulator {  
    public int FindMax(int num1, int num2) {  
        /* local variable declaration */  
        int result;  
        if (num1 > num2)  
            result = num1;  
        else  
            result = num2;  
        return result;  
    }  
    ...  
}
```

Calling Method:

using System;

namespace CalculatorApplication {

class NumberManipulator {

public int FindMax(int num1, int num2) {int result;

if (num1 > num2)

result = num1;

else

result = num2;

return result;}

static void Main(string[] args) {int a = 100;

int b = 200;

int ret;

NumberManipulator n = new NumberManipulator();

ret = n.FindMax(a, b);

Console.WriteLine("Max value is : {0}", ret);

Console.ReadLine();

} }

Defining a Class

- A class definition starts with the keyword `class` followed by the class name; and the class body enclosed by a pair of curly braces. Following is the general form of a class definition –

```
<access specifier> class class_name {  
    // member variables  
    <access specifier> <data type> variable1;  
    <access specifier> <data type> variable2;  
    ...  
    <access specifier> <data type> variableN;  
    // member methods  
    ...  
    <access specifier> <return type> methodN(parameter_list) {  
        // method body  
    }  
}
```

Namespace and Assemblies

- A namespace is designed for providing a way to keep one set of names separate from another. The class names declared in one namespace does not conflict with the same class names declared in another.
- A namespace definition begins with the keyword namespace followed by the namespace name as follows –

```
namespace namespace_name {  
    // code declarations  
}
```

- To call the namespace-enabled version of either function or variable, prepend the namespace name as follows –

```
namespace_name.item_name;
```

```
using System;
namespace first_space {
    class namespace_cl {
        public void func() {Console.WriteLine("Inside first_space"); }
    }
}
namespace second_space {
    class namespace_cl {
        public void func() {Console.WriteLine("Inside second_space");}
    }
}
class TestClass {
    static void Main(string[] args) {
        first_space.namespace_cl fc = new first_space.namespace_cl();
        second_space.namespace_cl sc = new second_space.namespace_cl();
        fc.func();
        sc.func();
        Console.ReadKey();
    }
}
```

- An Assembly is a basic building block of .Net Framework applications. It is basically a compiled code that can be executed by the CLR. An assembly is a collection of types and resources that are built to work together and form a logical unit of functionality. An Assembly can be a DLL or exe depending upon the project that we choose.
- Assemblies are basically the following two types:

Private Assembly

Shared Assembly

1. Private Assembly

- It is an assembly that is being used by a single application only. Suppose we have a project in which we refer to a DLL(Dynamic Link Libraries) so when we build that project that DLL will be copied to the bin folder of our project. That DLL becomes a private assembly within our project. Generally, the DLLs that are meant for a specific project are private assemblies.

2. Shared Assembly

- Assemblies that can be used in more than one project are known to be a shared assembly. Shared assemblies are generally installed in the GAC(Global Assembly Cache). Assemblies that are installed in the GAC are made available to all the .Net applications on that machine.

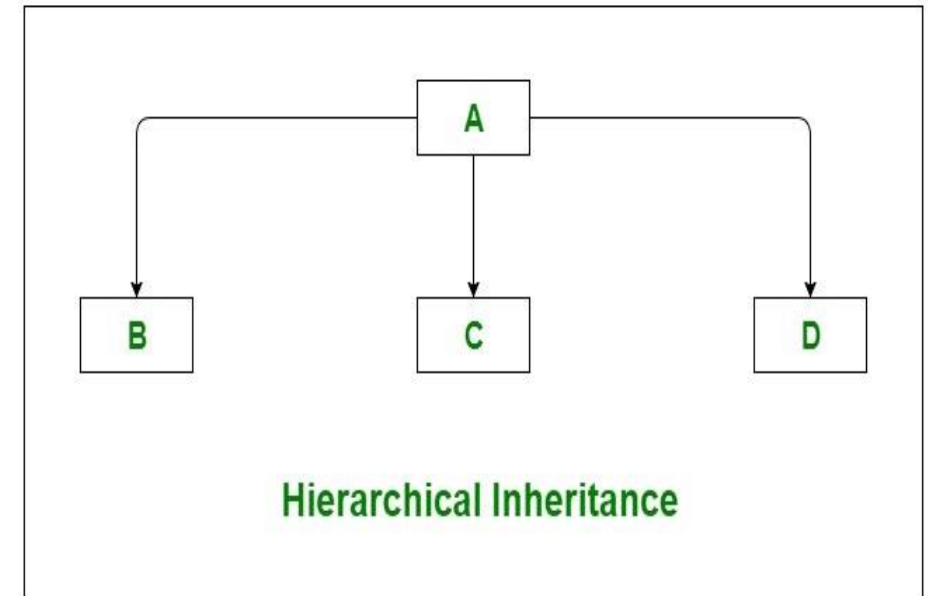
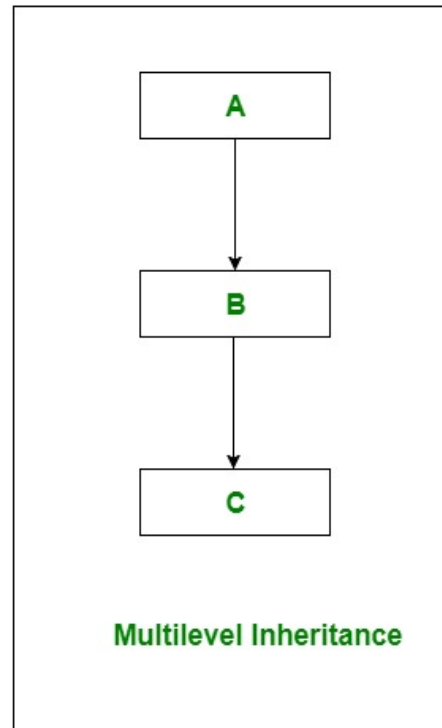
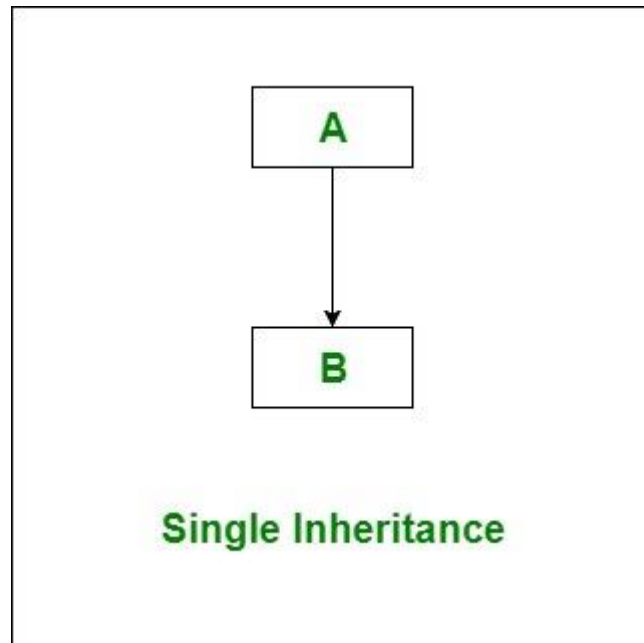
Inheritance

- Inheritance is an important pillar of OOP(Object Oriented Programming). It is the mechanism in C# by which one class is allowed to inherit the features(fields and methods) of another class.

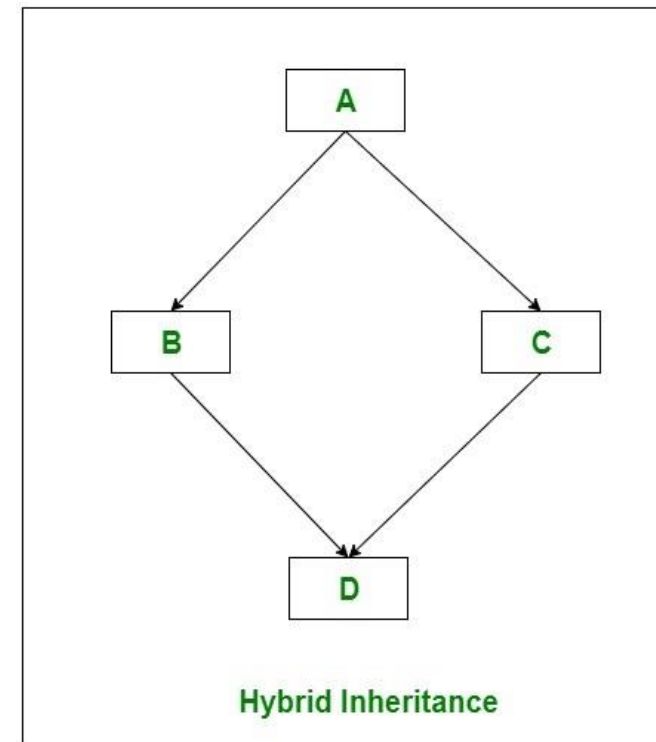
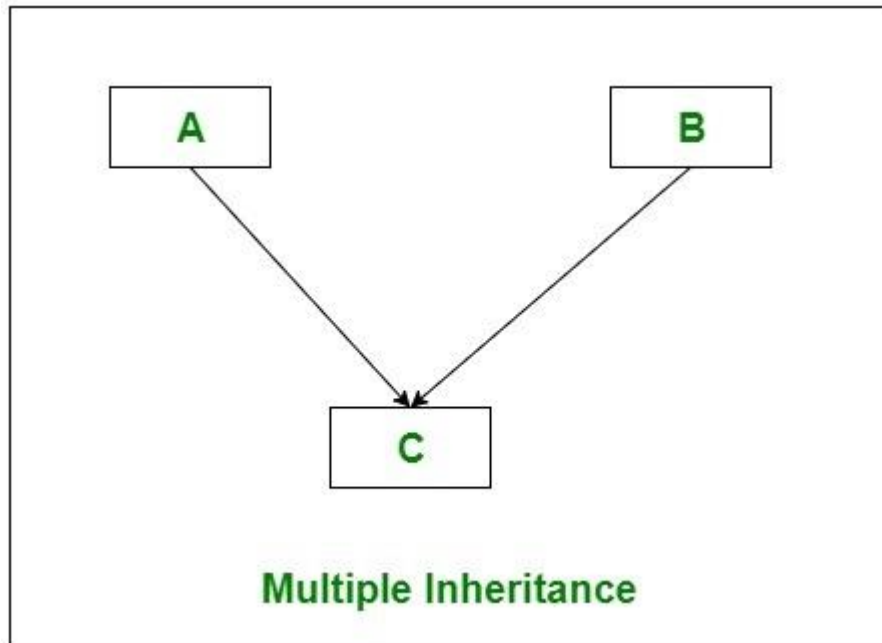
Important terminology:

- Super Class: The class whose features are inherited is known as super class(or a base class or a parent class).
- Sub Class: The class that inherits the other class is known as subclass(or a derived class, extended class, or child class). The subclass can add its own fields and methods in addition to the superclass fields and methods.
- Reusability: Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

- **Single Inheritance:** In single inheritance, subclasses inherit the features of one superclass. In image below, the class A serves as a base class for the derived class B.
- **Multilevel Inheritance:** In Multilevel Inheritance, a derived class will be inheriting a base class and as well as the derived class also act as the base class to other class. In below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C.
- **Hierarchical Inheritance:** In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In below image, class A serves as a base class for the derived class B, C, and D.



- Multiple Inheritance(Through Interfaces):In Multiple inheritance, one class can have more than one superclass and inherit features from all parent classes. Please note that C# does not support multiple inheritance with classes. In C#, we can achieve multiple inheritance only through Interfaces. In the image below, Class C is derived from interface A and B.
- Hybrid Inheritance(Through Interfaces): It is a mix of two or more of the above types of inheritance. Since C# doesn't support multiple inheritance with classes, the hybrid inheritance is also not possible with classes. In C#, we can achieve hybrid inheritance only through Interfaces.



ASP.NET

- ASP.NET is a web development platform, which provides a programming model, a comprehensive software infrastructure and various services required to build up robust web applications for PC, as well as mobile devices.
- ASP.NET works on top of the HTTP protocol, and uses the HTTP commands and policies to set a browser-to-server bilateral communication and cooperation.
- ASP.NET is a part of Microsoft .Net platform. ASP.NET applications are compiled codes, written using the extensible and reusable components or objects present in .Net framework. These codes can use the entire hierarchy of classes in .Net framework.
- The ASP.NET application codes can be written in any of the following languages:
 - C#
 - Visual Basic.Net
 - Jscript
 - J#
- ASP.NET is used to produce interactive, data-driven web applications over the internet. It consists of a large number of controls such as text boxes, buttons, and labels for assembling, configuring, and manipulating code to create HTML pages

Anatomy of a Web Form - Page Directive, Doctype

- An ASP.NET page is made up of a number of server controls along with HTML controls, text, and images.
- Sensitive data from the page and the states of different controls on the page are stored in hidden fields that form the context of that page request.
- An ASP.NET page is also a server side file saved with the .aspx extension.
- It is modular in nature and can be divided into the following core sections:
 1. Page Directives
 2. Code Section
 3. Page Layout

1. Page Directives

- The page directives set up the environment for the page to run. The @Page directive defines page-specific attributes used by ASP.NET page parser and compiler. Page directives specify how the page should be processed, and which assumptions need to be taken about the page.
- It allows importing namespaces, loading assemblies, and registering new controls with custom tag names and namespace prefixes.

2. Code Section

- The code section provides the handlers for the page and control events along with other functions required. We mentioned that, ASP.NET follows an object model. Now, these objects raise events when some events take place on the user interface, like a user clicks a button or moves the cursor. The kind of response these events need to reciprocate is coded in the event handler functions. The event handlers are nothing but functions bound to the controls.
- The code section or the code behind file provides all these event handler routines, and other functions used by the developer. The page code could be precompiled and deployed in the form of a binary assembly.

3. Page Layout

- The page layout provides the interface of the page. It contains the server controls, text, inline JavaScript, and HTML tags.

Adding Event Handlers

- An event is an action or occurrence such as a mouse click, a key press, mouse movements, or any system-generated notification. A process communicates through events.
- For example, interrupts are system-generated events. When events occur, the application should be able to respond to it and manage it.
- Events in ASP.NET raised at the client machine, and handled at the server machine.
- For example, a user clicks a button displayed in the browser. A Click event is raised. The browser handles this client-side event by posting it to the server.
- The server has a subroutine describing what to do when the event is raised; it is called the event-handler.
- Therefore, when the event message is transmitted to the server, it checks whether the Click event has an associated event handler. If it has, the event handler is executed.

- Event Arguments
- ASP.NET event handlers generally take two parameters and return void. The first parameter represents the object raising the event and the second parameter is event argument.
- The general syntax of an event is:

```
private void EventName (object sender, EventArgs e);
```

- Application and Session Events

The most important application events are:

1. Application_Start - It is raised when the application/website is started.
2. Application_End - It is raised when the application/website is stopped.
3. Similarly, the most used Session events are:
4. Session_Start - It is raised when a user first requests a page from the application.
5. Session_End - It is raised when the session ends.

- Page and Control Events

Common page and control events are:

1. DataBinding - It is raised when a control binds to a data source.
2. Disposed - It is raised when the page or the control is released.
3. Error - It is a page event, occurs when an unhandled exception is thrown.
4. Init - It is raised when the page or the control is initialized.
5. Load - It is raised when the page or a control is loaded.
6. PreRender - It is raised when the page or the control is to be rendered.
7. Unload - It is raised when the page or control is unloaded from memory.

The common control events are:

Event	Attribute	Controls
Click	OnClick	Button, image button, link button, image map
Command	OnCommand	Button, image button, link button
TextChanged	OnTextChanged	Text box
SelectedIndexChanged	OnSelectedIndexChanged	Drop-down list, list box, radio button list, check box list.
CheckedChanged	OnCheckedChanged	Check box, radio button

```
<% @Page Language="C#" %>
<script runat="server">
    private void convertoupper(object sender, EventArgs e)
    { string str = mytext.Value;
      changed_text.InnerHtml = str.ToUpper();}
</script>
<html>
    <head><title> Change to Upper Case </title></head>
    <body>
        <h3> Conversion to Upper Case </h3>
        <form runat="server">
            <input runat="server" id="mytext" type="text" />
            <input runat="server" id="button1" type="submit" value="Enter..."
OnServerClick="convertoupper"/>
            <hr />
            <h3> Results: </h3>
            <span runat="server" id="changed_text" />
        </form></body> </html>
```


ASP.NET File Type

- .asax --> It refers to the Global.asax file containing code that drives from the `HttpApplication` class. It resides Application root directory.
- .ascx --> It refers to a web user control file. It resides Application root directory or a subdirectory.
- .aspx --> It refers to a ASP.NET Web forms. It resides Application root directory or a subdirectory.
- .asmx --> It refers to an xml web services file that contains classes and methods. It resides Application root directory or a subdirectory.
- .axd --> It refers to a handler file that is used to website administration requests. It resides Application root directory.
- .cd --> It refers to a class diagram file. It resides Application root directory or a subdirectory.
- .compile --> It refers to a precompiled stub file that point to an assembly representing a compiled website file. For example .aspx, .ascx, .master file are precompiled. It resides Bin subdirectory.
- .browser --> It refers to a browser definition file used to identify the features of client browsers. It resides App_Browsers Subdirectory.
- .dll --> It refers to a compiled class library files (assembly file). It resides Bin subdirectory.
- .cs, .vb, .jsl --> It refers to a Source -code file that contains application logic. It resides App_Code Subdirectory or same directory as web page.

- .master --> It refers to a master page that defines the layout of web page in a web application. It resides Application root or subdirectory.
- .mdf,.sdf --> It refers to a SQL database file. It resides App_Data subdirectory.
- .mdb,ldb --> It refers to a Access database file. It resides App_Data subdirectory.
- .msgx,svc --> It refers to an indigo messaging framework (MFx) service file. It resides Application root or a subdirectory.
- .disco,.vsdisco --> It refers to a xml web services discovery file. It resides App_WebReferences subdirectory.
- .dsdgm, .dsprototype --> It refers to a distributed services diagram file that can be added to any visual studio solution. It resides Application root or a subdirectory.
- .licx, .webinfo --> It refers to a license file. It resides Application root or a subdirectory.
- .soap --> It refers to a SOAP extension file .It resides Application root or a subdirectory.
- .sitemap --> It refers to a site-map file that containing the structure of the website. It resides Application root directory.
- .rem --> It refers to a handler file which implements remoting concepts in web application. It resides Application root or a subdirectory.
- .resx , .resources --> it refers to a resource file that containing resource strings. It resides App_GlobalResources or App_LocalResources subdirectory.

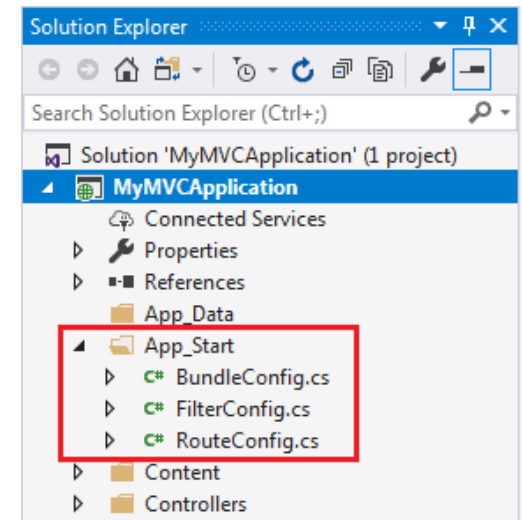
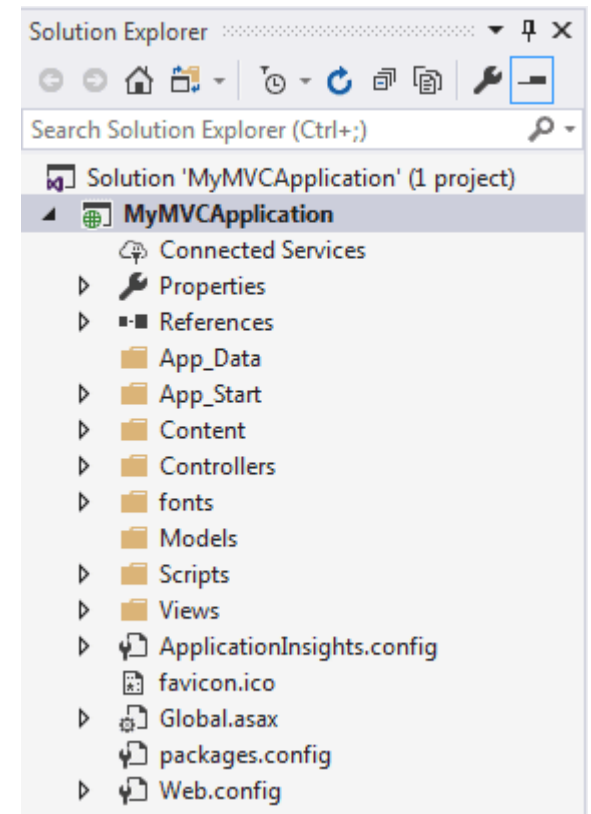
ASP.NET Web Folders

- App_Data

The App_Data folder can contain application data files like LocalDB, .mdf files, XML files, and other data related files. IIS will never serve files from App_Data folder.

- App_Start

The App_Start folder can contain class files that will be executed when the application starts. Typically, these would be config files like AuthConfig.cs, BundleConfig.cs, FilterConfig.cs, RouteConfig.cs etc. MVC 5 includes BundleConfig.cs, FilterConfig.cs and RouteConfig.cs by default. We will see the significance of these files later.



- Content

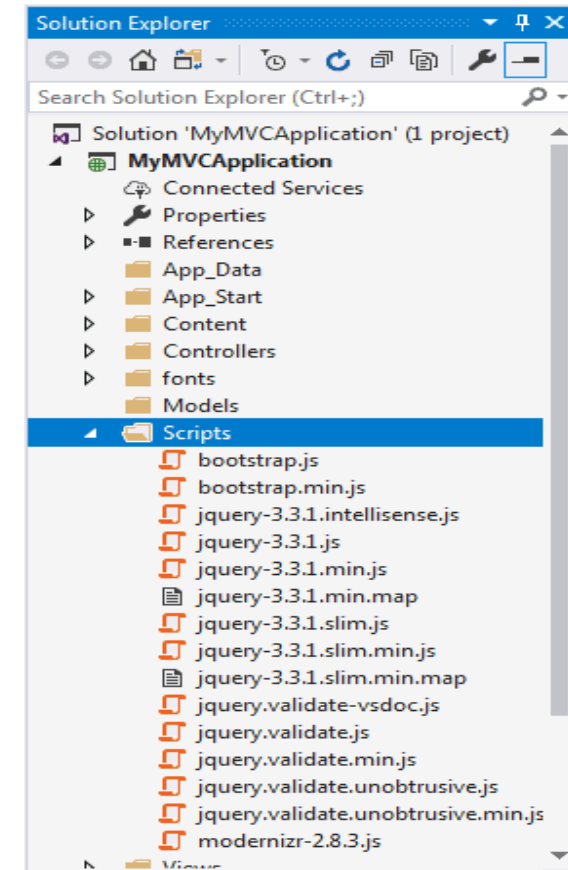
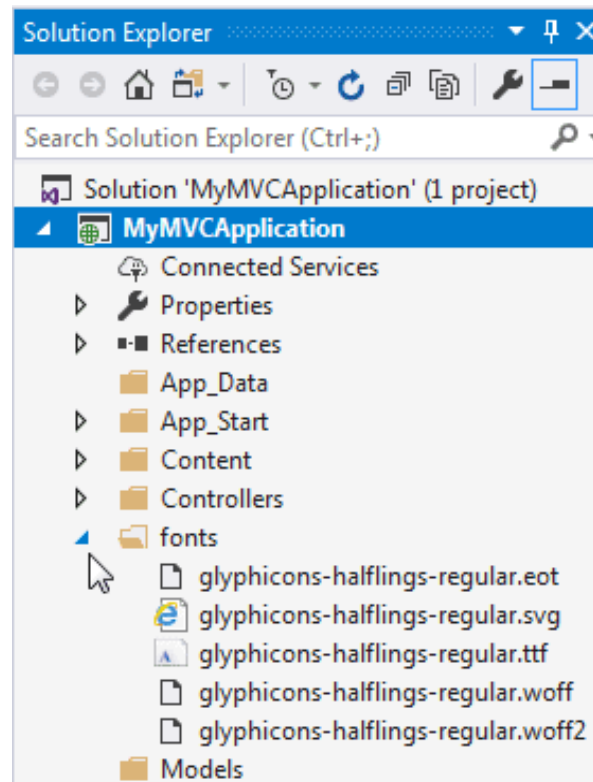
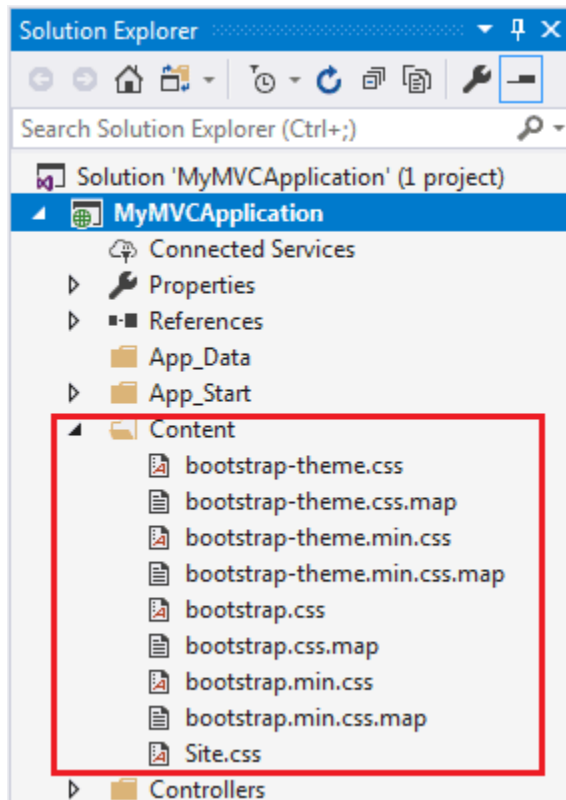
The Content folder contains static files like CSS files, images, and icons files. MVC 5 application includes bootstrap.css, bootstrap.min.css, and Site.css by default.

- fonts

The Fonts folder contains custom font files for your application.

- Scripts

The Scripts folder contains JavaScript or VBScript files for the application. MVC 5 includes javascript files for bootstrap, jquery 1.10, and modernizr by default.



- Global.asax

Global.asax file allows you to write code that runs in response to application-level events, such as `Application_BeginRequest`, `application_start`, `application_error`, `session_start`, `session_end`, etc.

- Packages.config

Packages.config file is managed by NuGet to track what packages and versions you have installed in the application.

- Web.config

Web.config file contains application-level configurations.

HTML Server Controls

View State

- An approach of saving data for the user, is the ViewState it allows ASP.NET to repopulate form fields on each postback to the server, making sure that a form is not automatically cleared when the user hits the submit button.
- All this happens automatically, unless you turn it off, but you can actually use the ViewState for your own purposes as well.
- Please keep in mind though, that while cookies and sessions can be accessed from all your pages on your website, ViewState values are not carried between pages.
- **Advantages of View State**
 1. Easy to Implement.
 2. No server resources are required: The View State is contained in a structure within the page load.
 3. Enhanced security features: It can be encoded and compressed or Unicode implementation.
- **Disadvantages of View State**
 1. Security Risk: The Information of View State can be seen in the page output source directly. You can manually encrypt and decrypt the contents of a Hidden Field, but It requires extra coding. If security is a concern then consider using a Server-Based state Mechanism so that no sensitive information is sent to the client.
 2. Performance: Performance is not good if we use a large amount of data because View State is stored in the page itself and storing a large value can cause the page to be slow.
 3. Device limitation: Mobile Devices might not have the memory capacity to store a large amount of View State data.
 4. It can store values for the same page only.

HTML Control Classes

- The `HtmlControl` object is pretty important to HTML server controls. Because every property and method it has is inherited by every HTML server control
- The HTML server controls have their own properties and methods, as well, but they all have the properties and methods contained in the `HtmlControl` object.

Property	Description
<code>Attribute</code>	Returns the object's attributes collection.
<code>Disabled</code>	A Boolean (true or false) value that you can get or set that indicates whether a control is disabled.
<code>EnableViewState</code>	A Boolean (true or false) value that you can get or set that indicates whether a control should maintain its viewstate.
<code>ID</code>	A string that you can get or set that defines the Identifier for the control.
<code>Style</code>	Returns the <code>CSSStyleCollection</code> for a control.
<code>TagName</code>	Returns the tag name of an element such as <code>input</code> or <code>div</code> .
<code>Visible</code>	A Boolean (true or false) value that you can get or set that indicates whether a control is rendered to HTML for delivery to the client's browser.

HtmlContainerControl Class

- HtmlContainerControl is used as the parent for any HTML control that requires a closing tag, such as div, form, or select.
- This class actually inherits all its properties and methods from the HtmlControl class and adds a few of its own. So to clarify, any object that is a container-type object doesn't directly inherit from the HtmlControl class.
- The HtmlContainerControl actually inherits the HtmlControl, and then when a container-type object uses the HtmlContainerControl it gets the HtmlControl class's objects that way.

Html InputControl Class

- Just like the HtmlContainerControl, the HtmlInput inherits from the HtmlControl and adds a few properties of its own for the different object that live off it. It brings three additional properties to the table

Property	Description
Name	Gets or sets the unique name for the <code>HtmlInput</code> control.
Type	Determines what kind of <code>Input</code> element the <code>HtmlInput</code> control is.
Value	Gets or sets the value of the content of the <code>HtmlInput</code> object.

global.asax File

- global.asax allows you to write code that runs in response to "system level" events, such as the application starting, a session ending, an application error occurring, without having to try and shoe-horn that code into each and every page of your site.
- Application_Init: Fired when an application initializes or is first called. It's invoked for all HttpApplication object instances.
- Application_Disposed: Fired just before an application is destroyed. This is the ideal location for cleaning up previously used resources.
- Application_Error: Fired when an unhandled exception is encountered within the application.
- Application_Start: Fired when the first instance of the HttpApplication class is created. It allows you to create objects that are accessible by all HttpApplication instances.
- Application_End: Fired when the last instance of an HttpApplication class is destroyed. It's fired only once during an application's lifetime.
- Application_AcquireRequestState: Fired when the ASP.NET page framework gets the current state (Session state) related to the current request.
- Application_ReleaseRequestState: Fired when the ASP.NET page framework completes execution of all event handlers. This results in all state modules to save their current state data.
- Application_ResolveRequestCache: Fired when the ASP.NET page framework completes an authorization request. It allows caching modules to serve the request from the cache, thus bypassing handler execution.
- Application_UpdateRequestCache: Fired when the ASP.NET page framework completes handler execution to allow caching modules to store responses to be used to handle subsequent requests.
- Application_AuthenticateRequest: Fired when the security module has established the current user's identity as valid. At this point, the user's credentials have been validated.
- Application_AuthorizeRequest: Fired when the security module has verified that a user can access resources.
- Session_Start: Fired when a new user visits the application Web site.
- Session_End: Fired when a user's session times out, ends, or they leave the application Web site.

web.config File

- A configuration file (web.config) is used to manage various settings that define a website.
- The settings are stored in XML files that are separate from your application code. In this way you can configure settings independently from your code.
- Generally a website contains a single Web.config file stored inside the application root directory. However there can be many configuration files that manage settings at various levels within an application.
- ASP.NET Configuration system is used to describe the properties and behaviors of various aspects of ASP.NET applications.
- Configuration files help you to manage the many settings related to your website.
- Each file is an XML file (with the extension .config) that contains a set of configuration elements. Configuration information is stored in XML-based text files.

Benefits of XML-based Configuration files

- ASP.NET Configuration system is extensible and application specific information can be stored and retrieved easily. It is human readable.
- You need not restart the web server when the settings are changed in configuration file. ASP.NET automatically detects the changes and applies them to the running ASP.NET application.
- You can use any standard text editor or XML parser to create and edit ASP.NET configuration files.

What Web.config file contains?

- There are number of important settings that can be stored in the configuration file. Some of the most frequently used configurations, stored conveniently inside Web.config file are:
 1. Database connections
 2. Caching settings
 3. Session States
 4. Error Handling
 5. Security

.NET Technologies

SIUSCS46

UNIT 2

By Shivani Deopa

Web Controls:

Web Controls:

- The Web controls reside in the `System.Web.UI.WebControls` namespace, which is available to all Web Forms pages automatically.
- They provide a range of functionality, from simple data entry to complex data validation.
- **Two classes are considered to be base classes of ASP.NET server controls: `System.Web.UI.Control` and `System.Web.UI.WebControls.WebControl`.**
- Web controls fall into eight categories: input, display, action, selection, databound, rich, validation
- They have properties, methods, and events that can be accessed at run time from code running on the server.
- Controls are small building blocks of the graphical user interface, which include text boxes, buttons, check boxes, list boxes, labels, and numerous other tools. Using these tools, the users can enter data, make selections and indicate their preferences.
- Controls are also used for structural jobs, like validation, data access, security, creating master pages, and data manipulation.

- ASP.NET server controls are the primary controls used in ASP.NET. These controls can be grouped into the following categories:
 1. **Validation controls** - These are used to validate user input and they work by running client-side script.
 2. **Data source controls** - These controls provides data binding to different data sources.
 3. **Data view controls** - These are various lists and tables, which can bind to data from data sources for displaying.
 4. **Personalization controls** - These are used for personalization of a page according to the user preferences, based on user information.
 5. **Login and security controls** - These controls provide user authentication.
 6. **Master pages** - These controls provide consistent layout and interface throughout the application.
 7. **Navigation controls** - These controls help in navigation. For example, menus, tree view etc.
 8. **Rich controls** - These controls implement special features. For example, AdRotator, FileUpload, and Calendar control.
- In addition, visual studio has the following features, to help produce in error-free coding:
 - a. Dragging and dropping of controls in design view
 - b. IntelliSense feature that displays and auto-completes the properties
 - c. The properties window to set the property values directly

List Controls

- ASP.NET provides the following list controls.
 1. Drop-down list
 2. List box
 3. Radio button list
 4. Check box list
 5. Bulleted list
- These controls display list of options to select. You can select one or more options, the choice depends upon control.
- They all derive from the `System.Web.UI.WebControls.ListControl` class

- Some of the important common properties of list controls are as follows:
 1. SelectedValue: Get the value of the selected item from the dropdown list.
 2. SelectedIndex: Gets the index of the selected item from the dropdown box.
 3. SelectedItem: Gets the text of selected item from the list.
 4. Items: Gets the collection of items from the dropdown list.
 5. DataTextField: Name of the data source field to supply the text of the items. Generally this field came from the datasource.
 6. DataValueField: Name of the data source field to supply the value of the items. This is not visible field to list controls, but you can use it in the code.
 7. DataSourceID: ID of the datasource control to provide data.
- There are several ways through which you can populate these controls such as:
 - a. By using data from database.
 - b. Directly write code to add items.
 - c. Add items through the items collection from property window.
 - d. Write HTML to populate these controls.
 - e. Use inbuilt datasource controls.

<https://www.tutorialride.com/asp-net/list-controls-in-asp-net.htm>

Table Controls

- In the .NET Framework the Table class enables you to build an HTML table.
- The System.Web.UI.Controls namespace defines the Table class, along with the other Web controls.
- You can create tables in .NET using a Table control and its helper controls TableRow and TableCell.
- As with all Web controls, you can create a Table control at run-time as well as at design-time using the VS .NET IDE

- Table: ASP.NET table and its Helper Control Classes
Manages a collection of table cells such as adding a cell to a row or removing a cell from it.

CONTROL	CODE	DESCRIPTION
Table	<asp:Table>	The System.Web.UI.Table class encapsulates an HTML table. An HTML table control, used to creates a table with the help of TableRow and TableCell.
TableRow	<asp:TableRow>	The System.Web.UI.TableRow class encapsulates a row within a table, which later can be used to get or set row's cells values using TableCell.
TableCell	<asp:TableCell>	The System.Web.UI.TableCell class encapsulates a cell within a table.
TableRowCollection	<asp:TableRowCollection>	The System.Web.UI.TableCell class encapsulates a TableRowCollection and is used to manage a collection of table a collection or removing a row from it.
TableCellCollection	<asp:TableCellCollection>	Manages a collection of table cells such as adding a cell to a row or removing a cell from it.
TableHeaderCollection	<asp:TableHeaderCell>	Encapsulate a table header cell.

Web Control Events and AutoPostBack

- Web controls are basically HTML elements wrapped under easy to use scripting tags of ASP+ and provide rich functionality in your FORMs or pages. Web controls range from simple text box to advance grids and lists. Some of the controls like data grid can be bound to data sources much like Visual Basic data bound controls.
- What are advantages of web controls ?
 1. They are highly customizable and provide common properties and methods throughout.
 2. Even though web controls provide rich functionality finally they are rendered as plain HTML in the client browser thus avoiding browser compatibility problems.
 3. They can be used in object oriented manner much like Visual Basic controls.
 4. They are processed at server side. This makes ASP+ page much readable and cleaner as compared to traditional ASP page.
 5. They can be data bound
 6. State maintenance is automatically taken care by ASP+

- What are common web controls available?
- Following is a list of common web controls :

Label, Panel, Button, TextBox, CheckBox, RadioButton, RadioButtonList, ListBox, DropDownList, Table/TableRow/TableCell, DataGrid, DataList, Repeater, Calender, Validation controls

- How do I handle Web control events?

Web control are processed at server side. This means that all the events generated by a control will be handled at server end. The event handling has changed with ASP+. Each event handler now has following syntax :

```
public sub myeventhandler(source as Object, evt as EventArgs)  
'access your web controls here and  
'manipulate their properties  
end sub
```

Here, source is the object which caused the event to be raised and evt is an object providing more information about the event.

Control Name	Applicable Events	Description
Label	None	It is used to display text on the HTML page.
TextBox	TextChanged	It is used to create a text input in the form.
Button	Click, Command	It is used to create a button.
LinkButton	Click, Command	It is used to create a button that looks similar to the hyperlink.
ImageButton	Click	It is used to create an imagesButton. Here, an image works as a Button.
Hyperlink	None	It is used to create a hyperlink control that responds to a click event.
DropDownList	SelectedIndexChanged	It is used to create a dropdown list control.
ListBox	SelectedIndexCnhaged	It is used to create a ListBox control like the HTML control.

CheckBox	CheckChanged	It is used to create checkbox.
CheckBoxList	SelectedIndexChanged	It is used to create a group of check boxes that all work together.
RadioButton	CheckChanged	It is used to create radio button.
RadioButtonList	SelectedIndexChanged	It is used to create a group of radio button controls that all work together.
Image	None	It is used to show image within the page.
Panel	None	It is used to create a panel that works as a container.
Placeholder	None	It is used to set placeholder for the control.
Calendar	SelectionChanged, VisibleMonthChanged, DayRender	It is used to create a calendar. We can set the default date, move forward and backward etc.
AdRotator	AdCreated	It allows us to specify a list of ads to display. Each time the user re-displays the page.
Table	None	It is used to create table.

- Autopostback is the mechanism, by which the page will be posted back to the server automatically based on some events in the web controls. In some of the web controls, the property called auto post back, which if set to true, will send the request to the server when an event happens in the control
- This is the event which is default associate with any webcontrol. For example, in case of Button click event, in case of Check box CheckChangedEvent is there. So in case of AutoPostBack true these events are called by default and event handle at server side.

ASP .NET-Life Cycle

- ASP.NET life cycle specifies, how:
- ASP.NET processes pages to produce dynamic output
- The application and its pages are instantiated and processed
- ASP.NET compiles the pages dynamically
- The ASP.NET life cycle could be divided into two groups:
 1. Application Life Cycle
 2. Page Life Cycle

ASP.NET Application Life Cycle

- The application life cycle has the following stages:
- User makes a request for accessing application resource, a page. Browser sends this request to the web server.
- A unified pipeline receives the first request and the following events take place:
 - An object of the class `ApplicationManager` is created.
 - An object of the class `HostingEnvironment` is created to provide information regarding the resources.
 - Top level items in the application are compiled.
- Response objects are created. The application objects such as `HttpContext`, `HttpRequest` and `HttpResponse` are created and initialized.
- An instance of the `HttpApplication` object is created and assigned to the request.
- The request is processed by the `HttpApplication` class. Different events are raised by this class for processing the request.

ASP.NET Page Life Cycle

- When a page is requested, it is loaded into the server memory, processed, and sent to the browser. Then it is unloaded from the memory. At each of these steps, methods and events are available, which could be overridden according to the need of the application. In other words, you can write your own code to override the default code.
- The Page class creates a hierarchical tree of all the controls on the page. All the components on the page, except the directives, are part of this control tree. You can see the control tree by adding `trace= "true"` to the page directive. We will cover page directives and tracing under 'directives' and 'event handling'.
- The page life cycle phases are:
 - Initialization
 - Instantiation of the controls on the page
 - Restoration and maintenance of the state
 - Execution of the event handler codes
 - Page rendering

- Following are the different stages of an ASP.NET page:

Page request - When ASP.NET gets a page request, it decides whether to parse and compile the page, or there would be a cached version of the page; accordingly the response is sent.

Starting of page life cycle - At this stage, the Request and Response objects are set. If the request is an old request or post back, the IsPostBack property of the page is set to true. The UICulture property of the page is also set.

Page initialization - At this stage, the controls on the page are assigned unique ID by setting the UniqueID property and the themes are applied. For a new request, postback data is loaded and the control properties are restored to the view-state values.

Page load - At this stage, control properties are set using the view state and control state values.

Validation - Validate method of the validation control is called and on its successful execution, the IsValid property of the page is set to true.

Postback event handling - If the request is a postback (old request), the related event handler is invoked.

Page rendering - At this stage, view state for the page and all controls are saved. The page calls the Render method for each control and the output of rendering is written to the OutputStream class of the Response property of page.

Unload - The rendered page is sent to the client and page properties, such as Response and Request, are unloaded and all cleanup done.

- Following are the page life cycle events:

PreInit - PreInit is the first event in page life cycle. It checks the IsPostBack property and determines whether the page is a postback. It sets the themes and master pages, creates dynamic controls, and gets and sets profile property values. This event can be handled by overloading the OnPreInit method or creating a Page_PreInit handler.

Init - Init event initializes the control property and the control tree is built. This event can be handled by overloading the OnInit method or creating a Page_Init handler.

InitComplete - InitComplete event allows tracking of view state. All the controls turn on view-state tracking.

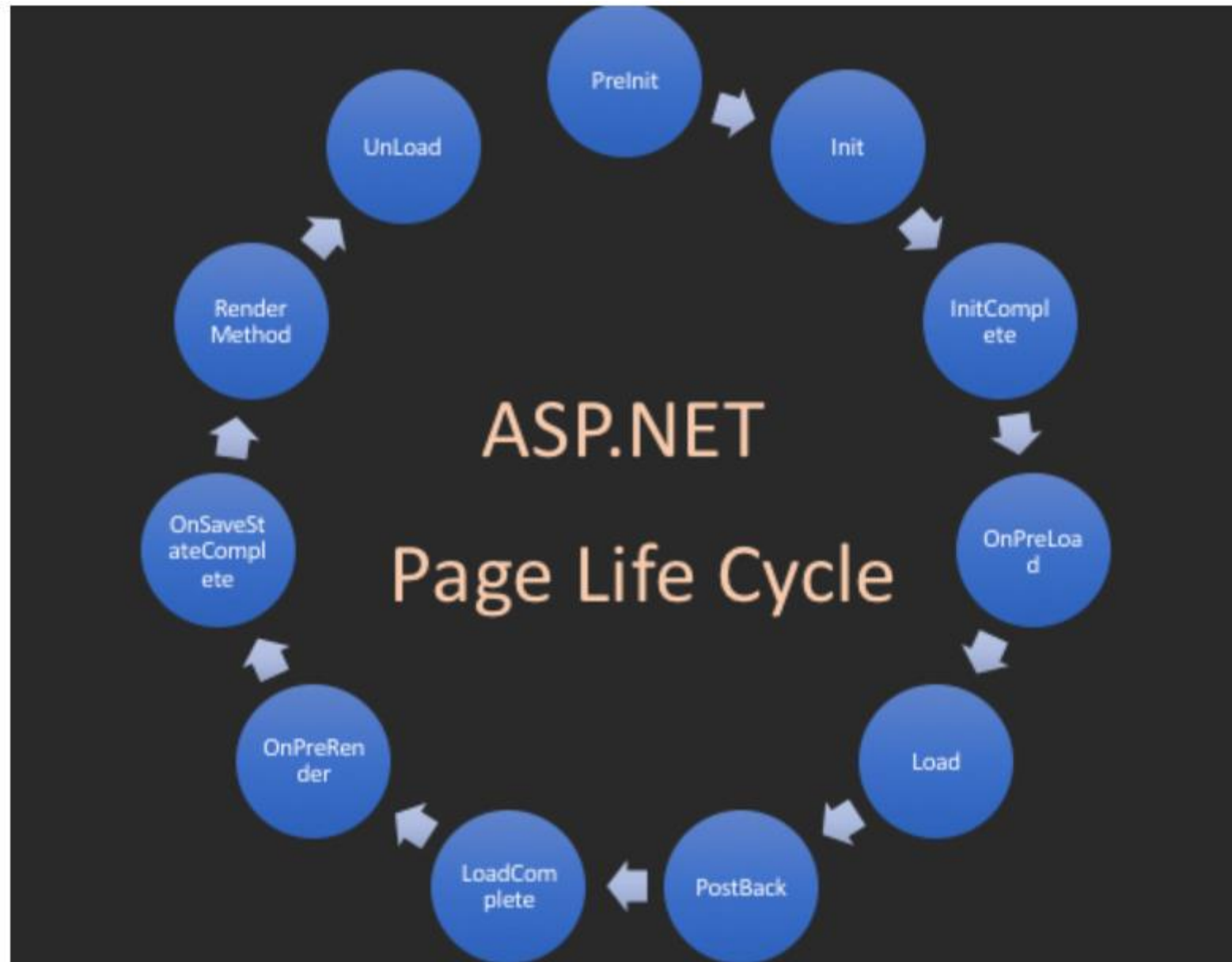
LoadViewState - LoadViewState event allows loading view state information into the controls.

LoadPostData - During this phase, the contents of all the input fields are defined with the <form> tag are processed.

PreLoad - PreLoad occurs before the post back data is loaded in the controls. This event can be handled by overloading the OnPreLoad method or creating a Page_PreLoad handler.

Load - The Load event is raised for the page first and then recursively for all child controls. The controls in the control tree are created. This event can be handled by overloading the OnLoad method or creating a Page_Load handler.

- **LoadComplete** - The loading process is completed, control event handlers are run, and page validation takes place. This event can be handled by overloading the OnLoadComplete method or creating a Page_LoadComplete handler
- **PreRender** - The PreRender event occurs just before the output is rendered. By handling this event, pages and controls can perform any updates before the output is rendered.
- **PreRenderComplete** - As the PreRender event is recursively fired for all child controls, this event ensures the completion of the pre-rendering phase.
- **SaveStateComplete** - State of control on the page is saved. Personalization, control state and view state information is saved. The HTML markup is generated. This stage can be handled by overriding the Render method or creating a Page_Render handler.
- **Unload** - The UnLoad phase is the last phase of the page life cycle. It raises the UnLoad event for all controls recursively and lastly for the page itself. Final cleanup is done and all resources and references, such as database connections, are freed. This event can be handled by modifying the OnUnload method or creating a Page_UnLoad handler.



Rich Controls:

Rich Controls - Calendar Control

- The calendar control is a functionally rich web control, which provides the following capabilities:
 1. Displaying one month at a time
 2. Selecting a day, a week or a month
 3. Selecting a range of days
 4. Moving from month to month
 5. Controlling the display of the days programmatically
- The basic syntax of a calendar control is:
`<asp:Calendar ID="Calendar1" runat="server">`

- The Calendar control has the following three most important events and properties that allow the developers to program the calendar control. They are:

Events	Description
SelectionChanged	It is raised when a day, a week or an entire month is selected.
DayRender	It is raised when each data cell of the calendar control is rendered.
VisibleMonthChanged	It is raised when user changes a month.

Properties	Description
Day	To select a single day.
DayWeek	To select a single day or an entire week.
DayWeekMonth	To select a single day, a week, or an entire month.
None	Nothing can be selected.

AdRotator Control

- The AdRotator control randomly selects banner graphics from a list, which is specified in an external XML schedule file. This external XML schedule file is called the advertisement file.
- The AdRotator control allows you to specify the advertisement file and the type of window that the link should follow in the AdvertisementFile and the Target property respectively.

```
<asp:AdRotator ID="AdRotator1" runat="server" AdvertisementFile ="xmlfilename"  
Target="_blank" />
```

- The advertisement file is an XML file, which contains the information about the advertisements to be displayed.
- Extensible Markup Language (XML) is a W3C standard for text document markup. It is a text-based markup language that enables you to store data in a structured format by using meaningful tags.
- The term 'extensible' implies that you can extend your ability to describe a document by defining meaningful tags for the application.

- Like all XML files, the advertisement file needs to be a structured text file with well-defined tags delineating the data. There are the following standard XML elements that are commonly used in the advertisement file:

Element	Description
Advertisements	Encloses the advertisement file.
Ad	Delineates separate ad.
ImageUrl	The path of image that will be displayed.
NavigateUrl	The link that will be followed when the user clicks the ad.
AlternateText	The text that will be displayed instead of the picture if it cannot be displayed.
Keyword	Keyword identifying a group of advertisements. This is used for filtering.
Impressions	The number indicating how often an advertisement will appear.
Height	Height of the image to be displayed.
Width	Width of the image to be displayed.

- Properties and Event of AdRotator are:

Events	Description
AdCreated	It is raised once per round trip to the server after creation of the control, but before the page is rendered
DataBinding	Occurs when the server control binds to a data source.
DataBound	Occurs after the server control binds to a data source.
Disposed	Occurs when a server control is released from memory, which is the last stage of the server control lifecycle when an ASP.NET page is requested
Init	Occurs when the server control is initialized, which is the first step in its lifecycle.
Load	Occurs when the server control is loaded into the Page object.
PreRender	Occurs after the Control object is loaded but prior to rendering.
Unload	Occurs when the server control is unloaded from memory.

Properties	Description
AdvertisementFile	The path to the advertisement file.
AlternateTextFeild	The element name of the field where alternate text is provided. The default value is AlternateText.
DataMember	The name of the specific list of data to be bound when advertisement file is not used.
DataSource	Control from where it would retrieve data.
DataSourceID	Id of the control from where it would retrieve data.
Font	Specifies the font properties associated with the advertisement banner control.
ImageUrlField	The element name of the field where the URL for the image is provided. The default value is ImageUrl.
KeywordFilter	For displaying the keyword based ads only.
NavigateUrlField	The element name of the field where the URL to navigate to is provided. The default value is NavigateUrl.

MultiView Control

- MultiView and View controls allow you to divide the content of a page into different groups, displaying only one group at a time. Each View control manages one group of content and all the View controls are held together in a MultiView control.
- The MultiView control is responsible for displaying one View control at a time. The View displayed is called the active view.

```
<asp:MultiView ID= "MultiView1" runat= "server">  
    <asp:View ID= "View1" runat= "server"> </asp:View>  
</asp:MultiView>
```

- Properties of View and MultiView Controls
- Both View and MultiView controls are derived from Control class and inherit all its properties, methods, and events. The most important property of the View control is Visible property of type Boolean, which sets the visibility of a view.
- The MultiView control has the following important properties:

Properties	Description
Views	Collection of View controls within the MultiView.
ActiveViewIndex	A zero based index that denotes the active view. If no view is active, then the index is -1.

The following table shows the default command names of the above properties:

Properties	Description
NextViewCommandName	NextView
PreviousViewCommandName	PrevView
SwitchViewByIDCommandName	SwitchViewByID
SwitchViewByIndexCommandName	SwitchViewByIndex

The important methods of the multiview control are:

Methods	Description
SetActiveview	Sets the active view
GetActiveview	Retrieves the active view

- Every time a view is changed, the page is posted back to the server and a number of events are raised. Some important events are:

Events	Description
ActiveViewChanged	Raised when a view is changed
Activate	Raised by the active view
Deactivate	Raised by the inactive view

Apart from the above mentioned properties, methods and events, multiview control inherits the members of the control and object class.

Validation:

Server-Side and Client-Side Validation

- Validations can be performed on the server side or on the client side (web browser). The user input validation take place on the Server Side during a post back session is called Server Side Validation and the user input validation take place on the Client Side (web browser) is called Client Side Validation.
- Client Side Validation does not require a postback. If the user request requires server resources to validate the user input, you should use Server Side Validation. If the user request does not require any server resources to validate the input , you can use Client Side Validation.
- **Server Side Validation**
- In the Server Side Validation, the input submitted by the user is being sent to the server and validated using one of server side scripting languages such as ASP.Net, PHP etc. After the validation process on the Server Side, the feedback is sent back to the client by a new dynamically generated web page. It is better to validate user input on Server Side because you can protect against the malicious users, who can easily bypass your Client Side scripting language and submit dangerous input to the server.
- **Client Side Validation**
- In the Client Side Validation you can provide a better user experience by responding quickly at the browser level. When you perform a Client Side Validation, all the user inputs validated in the user's browser itself. Client Side validation does not require a round trip to the server, so the network traffic which will help your server perform better. This type of validation is done on the browser side using script languages such as JavaScript, VBScript or HTML5 attributes.
- For example, if the user enter an invalid email format, you can show an error message immediately before the user move to the next field, so the user can correct every field before they submit the form.

Validation: Validation Controls

- ASP.NET validation controls validate the user input data to ensure that useless, unauthenticated, or contradictory data don't get stored.
- ASP.NET provides the following validation controls:
 1. RequiredFieldValidator
 2. RangeValidator
 3. CompareValidator
 4. RegularExpressionValidator
 5. CustomValidator
 6. ValidationSummary

- BaseValidator Class
- The validation control classes are inherited from the BaseValidator class hence they inherit its properties and methods. Therefore, it would help to take a look at the properties and the methods of this base class, which are common for all the validation controls:

Members	Description
ControlToValidate	Indicates the input control to validate.
Display	Indicates how the error message is shown.
EnableClientScript	Indicates whether client side validation will take.
Enabled	Enables or disables the validator.
ErrorMessage	Indicates error string.
Text	Error text to be shown if validation fails.
IsValid	Indicates whether the value of the control is valid.
SetFocusOnError	It indicates whether in case of an invalid control, the focus should switch to the related input control.
ValidationGroup	The logical group of multiple validators, where this control belongs.
Validate()	This method revalidates the control and updates the IsValid property.

RequiredFieldValidator Control

- The RequiredFieldValidator control ensures that the required field is not empty. It is generally tied to a text box to force input into the text box.
- The syntax of the control is as given:

```
<asp:RequiredFieldValidator ID="rfvcandidate"  
    runat="server" ControlToValidate ="ddlcandidate"  
    ErrorMessage="Please choose a candidate"  
    InitialValue="Please choose a candidate">  
</asp:RequiredFieldValidator>
```

RangeValidator Control

- The RangeValidator control verifies that the input value falls within a predetermined range.
- It has three specific properties:

Properties	Description
Type	It defines the type of the data. The available values are: Currency, Date, Double, Integer, and String.
MinimumValue	It specifies the minimum value of the range.
MaximumValue	It specifies the maximum value of the range.

- The syntax of the control is as given:

```
<asp:RangeValidator ID="rvclass" runat="server" ControlToValidate="txtclass"
  ErrorMessage="Enter your class (6 - 12)" MaximumValue="12"
  MinimumValue="6" Type="Integer">
</asp:RangeValidator>
```


CompareValidator Control

- The CompareValidator control compares a value in one control with a fixed value or a value in another control.
- It has the following specific properties:

Properties	Description
Type	It specifies the data type.
ControlToCompare	It specifies the value of the input control to compare with.
ValueToCompare	It specifies the constant value to compare with.
Operator	It specifies the comparison operator, the available values are: Equal, NotEqual, GreaterThan, GreaterThanEqual, LessThan, LessThanEqual, and DataTypeCheck.

- The basic syntax of the control is as follows:

```
<asp:CompareValidator ID="CompareValidator1" runat="server"
    ErrorMessage="CompareValidator">
</asp:CompareValidator>
```

CustomValidator

- The CustomValidator control allows writing application specific custom validation routines for both the client side and the server side validation.
- The client side validation is accomplished through the ClientValidationFunction property. The client side validation routine should be written in a scripting language, such as JavaScript or VBScript, which the browser can understand.
- The server side validation routine must be called from the control's ServerValidate event handler. The server side validation routine should be written in any .Net language, like C# or VB.Net.
- The basic syntax for the control is as given:

```
<asp:CustomValidator ID="CustomValidator1" runat="server"  
    ClientValidationFunction=.cvf_func. ErrorMessage="CustomValidator">  
</asp:CustomValidator>
```

ValidationSummary

- The ValidationSummary control does not perform any validation but shows a summary of all errors in the page. The summary displays the values of the ErrorMessage property of all validation controls that failed validation.
- The following two mutually inclusive properties list out the error message:
 1. ShowSummary : shows the error messages in specified format.
 2. ShowMessageBox : shows the error messages in a separate window.
- The syntax for the control is as given:

```
<asp:ValidationSummary ID="ValidationSummary1" runat="server"  
    DisplayMode = "BulletList" ShowSummary = "true" HeaderText="Errors:" />
```

RegularExpressionValidator

- The RegularExpressionValidator allows validating the input text by matching against a pattern of a regular expression. The regular expression is set in the ValidationExpression property.
- The following table summarizes the commonly used syntax constructs for regular expressions:

Character Escapes	Description
\b	Matches a backspace.
\t	Matches a tab.
\r	Matches a carriage return.
\v	Matches a vertical tab.
\f	Matches a form feed.
\n	Matches a new line.
\	Escape character.

Metacharacters	Description
.	Matches any character except \n.
[abcd]	Matches any character in the set.
[^abcd]	Excludes any character in the set.
[2-7a-mA-M]	Matches any character specified in the range.
\w	Matches any alphanumeric character and underscore.
\W	Matches any non-word character.
\s	Matches whitespace characters like, space, tab, new line etc.
\S	Matches any non-whitespace character.
\d	Matches any decimal character.
\D	Matches any non-decimal character.

- The syntax of the control is as given:

```
<asp:RegularExpressionValidator ID="string" runat="server"  
ErrorMessage="string"  
    ValidationExpression="string" ValidationGroup="string">  
</asp:RegularExpressionValidator>
```

State Management:

State Management: Cross-Page Posting

- ASP.NET by default, submits the form to the same page. Cross page posting is submitting the form to a different page.
- This is usually required when you are creating a multi page form to collect information from the user on each page.
- When moving from the source to the target page, the values of controls in the source page can be accessed in the target page.
- To use cross-page posting, you have to use thePostBackURL attribute to specify the page you want to post to.
- Follow these steps :

- **Step 1:** Create a new ASP.NET website called CrossPagePosting. By default, the website is created with a single webpage, Default.aspx. Right click the project in the Solution Explorer > Add New Item > Web Form. Keep the original name Default2.aspx and click 'Add'. The website will now contain two pages, Default.aspx and Default2.aspx.
- **Step 2:** On the source page, Default.aspx, drop a button on the form. Set the text of the button as 'TargetButton'. Set the 'PostBackUrl' property of a Button to the URL of the target page, Default2.aspx.

```
<asp:Button ID="Button1" runat="server" PostBackUrl="~/Default2.aspx" Text="TargetButton" /></div>
```

- **Step 3:** In the target page Default2.aspx, drop a label on the page from the toolbox.
- **Step 4:** In the Page_Load() of Default2.aspx, you can then access the 'PreviousPage' property to check if the page is being accessed as Cross Page postback.

```
protected void Page_Load(object sender, EventArgs e)
```

```
{    if (Page.PreviousPage != null) {    }    }
```

- **Step 5:** To retrieve values from the source page, you must access controls using the 'FindControl()' method of the 'PreviousPage'. We will be accessing the Text property of the Button control placed in Default.aspx.

```
protected void Page_Load(object sender, EventArgs e)
```

```
{    if (Page.PreviousPage != null)
    {    Button btn = (Button)(Page.PreviousPage.FindControl("button1"));
        Label1.Text = btn.Text;    }    }
```

- **Step 6:** In the Solution Explorer, right click Default.aspx > 'Set as Start Page'. Run the application and click on the button. As you can observe, the page is posted to Default2.aspx and the value containing the name of the button control gets displayed in the label.

Query String

- A query string is a collection of characters input to a computer or web browser. A Query String is helpful when we want to transfer a value from one page to another.
- When we need to pass content between the HTML pages or aspx Web Forms in the context of ASP.NET, a Query String is very easy to use and the Query String follows a separating character, usually a Question Mark (?).
- It is basically used for identifying data appearing after this separating symbol. A Query String Collection is used to retrieve the variable values in the HTTP query string.
- If we want to transfer a large amount of data then we can't use the Request.QueryString. Query Strings are also generated by form submission or can be used by a user typing a query into the address bar of the browsers.
- Query Strings are contained in request headers. A Query String collection is a parsed version of the QUERY_STRING variable in the Server Variables collection.
- It enable us to retrieve the QUERY_STRING variable by name. When we use parameters with Request.QueryString, the server parses the parameters sent to the request and returns the effective or specified data.
- Syntax of Query String

Request.QueryString(variable)[(index).count]

- Create two web pages WebForm1.aspx and WebForm2.aspx
- WebForm1.aspx has 2 textbox and 2 label as firstname and lastname and a button
- Below the the code for button click event in side WebForm1.aspx.cs page

```
protected void Button1_Click1(object sender, EventArgs e)  
{  
    Response.Redirect("WebForm2.aspx ?firstname=" + TextBox3.Text + "&lastname=" + TextBox4.Text);  
}
```

- WebForm2.aspx has a label
- Below the the code for button click event in side WebForm2.aspx.cs page

```
protected void Page_Load(object sender, EventArgs e)  
{  
    string firstname = Request.QueryString["firstname"];  
    string lastname = Request.QueryString["lastname"];  
    Label1.Text = "welcome" + firstname + " " + lastname;  
}
```

Cookies

- ASP.NET Cookie is a small bit of text that is used to store user-specific information. This information can be read by the web application whenever user visits the site.
- When a user requests for a web page, web server sends not just a page, but also a cookie containing the date and time. This cookie stores in a folder on the user's hard disk.
- When the user requests for the web page again, browser looks on the hard drive for the cookie associated with the web page. Browser stores separate cookie for each different sites user visited.
- Type of Cookies
 1. Persist Cookie - A cookie has not have expired time Which is called as Persist Cookie
 2. Non-Persist Cookie - A cookie has expired time Which is called as Non-Persist Cookie

- Cookie's common property

1. Domain => Which is used to associate cookies to domain.
2. Secure => We can enable secure cookie to set true(HTTPs).
3. Value => We can manipulate individual cookie.
4. Values => We can manipulate cookies with key/value pair.
5. Expires => Which is used to set expire date for the cookies.

- Advantages of Cookie

1. Its clear text so user can able to read it.
2. We can store user preference information on the client machine.
3. Its easy way to maintain.
4. Fast accessing.

- Disadvantages of Cookie

1. If user clear cookie information we can't get it back.
2. No security.
3. Each request will have cookie information with page.

Session State

- When a user connects to an ASP.NET website, a new session object is created. When session state is turned on, a new session state object is created for each new request. This session state object becomes part of the context and it is available through the page.
- Session state is generally used for storing application data such as inventory, supplier list, customer record, or shopping cart. It can also keep information about the user and his preferences, and keep the track of pending operations.
- Sessions are identified and tracked with a 120-bit SessionID, which is passed from client to server and back as cookie or a modified URL. The SessionID is globally unique and random.
- The session state object is created from the HttpSessionState class, which defines a collection of session state items.

- Properties and Methods of Session State

Properties	Description
SessionID	The unique session identifier.
Item(name)	The value of the session state item with the specified name. This is the default property of the HttpSessionState class.
Count	The number of items in the session state collection.
TimeOut	Gets and sets the amount of time, in minutes, allowed between requests before the session-state provider terminates the session.

Methods	Description
Add(name, value)	Adds an item to the session state collection.
Clear	Removes all the items from session state collection.
Remove(name)	Removes the specified item from the session state collection.
RemoveAll	Removes all keys and values from the session-state collection.
RemoveAt	Deletes an item at a specified index from the session-state collection.

Application State

- The ASP.NET application is the collection of all web pages, code and other files within a single virtual directory on a web server. When information is stored in application state, it is available to all the users.
- To provide for the use of application state, ASP.NET creates an application state object for each application from the `HTTPApplicationState` class and stores this object in server memory. This object is represented by class file `global.asax`.
- Application State is mostly used to store hit counters and other statistical data, global application data like tax rate, discount rate etc. and to keep the track of users visiting the site.

- The HttpSessionState class has the following Properties and Methods:

Properties	Description
Item(name)	The value of the application state item with the specified name. This is the default property of the HttpSessionState class.
Count	The number of items in the application state collection.

Methods	Description
Add(name, value)	Adds an item to the application state collection.
Clear	Removes all the items from the application state collection.
Remove(name)	Removes the specified item from the application state collection.
RemoveAll	Removes all objects from an HttpSessionState collection.
RemoveAt	Removes an HttpSessionState object from a collection by index.
Lock()	Locks the application state collection so only the current user can access it.
Unlock()	Unlocks the application state collection so all the users can access it.

Website Navigation:

Site Maps

- The SiteMapPath control basically is used to access web pages of the website from one webpage to another.
- It is a navigation control and displays the map of the site related to its web pages. This map includes the pages in the particular website and displays the name of those pages.
- You can click on that particular page in the Site Map to navigate to that page. We can say that the SiteMapPath control displays links for connecting to URLs of other pages.
- The SiteMapPath control uses a property called SiteMapProvider for accessing data from databases and it stores the information in a data source. The SiteMapProvider internally utilizes the **SiteMapProvider** abstract class defined under the **System.Web** namespace.
- The representation of the SiteMapPath control is as follows:

Root Node->Child Node

- Public Properties of SiteMapPath class

1. **ParentLevelsDisplayed:** It specifies the number of levels of parent nodes and then displays the control accordingly related to the currently displayed node.
2. **RenderCurrentNodeAsLink:** It specifies whether or not the site navigation node that represents the currently displayed page is rendered as a hyperlink.
3. **PathSeperator:** It specifies the string that displays the SiteMapPath nodes in the rendered navigation path.

- Style properties of the SiteMapPath class

1. **CurrentNodeStyle:** It specifies the style used for the display text for the current node.
2. **RootNodeStyle:** It specifies the style for the root node style text.
3. **NodeStyle:** It specifies the style used for the display text for all nodes in the site navigation path.

Menu Control

- The Menu control is used to display a menu in an ASP.NET Web page and is often used in combination with a SiteMapDataSource control for navigating a Web site.
- The Menu control supports the following features:
 1. Data binding that allows the control's menu items to be bound to hierarchical data sources.
 2. Site navigation through integration with the SiteMapDataSource control.
 3. Programmatic access to the Menu object model to dynamically create menus, populate menu items, set properties, and so on.
 4. Customizable appearance through themes, user-defined images, styles, and user-defined templates.
- When the user clicks a menu item, the Menu control can either navigate to a linked Web page or simply post back to the server.
- If the NavigateUrl property of a menu item is set, the Menu control navigates to the linked page; otherwise, it posts the page back to the server for processing. By default, a linked page is displayed in the same window or frame as the Menu control.
- To display the linked content in a different window or frame, use the Target property of the Menu control.

- The Menu control has two modes of display: static and dynamic.
- Static display means that the Menu control is fully expanded all the time. The entire structure is visible, and a user can click on any part.
- In a dynamically displayed menu, only the portions you specify are static, while their child menu items are displayed when the user holds the mouse pointer over the parent node.
- A Menu control is made up of a tree of menu items represented by MenuItem objects. Menu items at the top level (level 0) are called root menu items.
- A menu item that has a parent menu item is called a child menu item. All root menu items are stored in the Items collection.
- Child menu items are stored in a parent menu item's ChildItems collection. Each menu item has a Text and a Value property.
- The value of the Text property is displayed in the Menu control, while the Value property is used to store any additional data about the menu item, such as data passed to the postback event associated with the menu item.
- When clicked, a menu item can navigate to another Web page indicated by the NavigateUrl property.

TreeView Control

- The TreeView control is used to display hierarchical data, such as a table of contents or file directory, in a tree structure and supports the following features:
 1. Data binding that allows the nodes of the control to be bound to XML, tabular, or relational data.
 2. Site navigation through integration with the SiteMapDataSource control.
 3. Node text that can be displayed as either plain text or hyperlinks.
 4. Programmatic access to the TreeView object model to create trees, populate nodes, set properties, and so on dynamically.
 5. Client-side node population (on supported browsers).
 6. The ability to display a check box next to each node.
 7. Customizable appearance through themes, user-defined images, and styles.

- The TreeView control is made up of nodes. Each entry in the tree is called a node and is represented by a TreeNode object. Node types are defined as follows:
 1. A node that contains other nodes is called a parent node.
 2. The node that is contained by another node is called a child node.
 3. A node that has no children is called a leaf node.
 4. The node that is not contained by any other node but is the ancestor to all the other nodes is the root node.
- A node can be both a parent and a child, but root, parent, and leaf nodes are mutually exclusive. Several visual and behavioral properties of nodes are determined by whether a node is a root, child, or leaf node.
- Although a typical tree structure has only one root node, the TreeView control allows you to add multiple root nodes to your tree structure. This is useful when you want to display item listings without displaying a single root node, as in a list of product categories.
- Each node has a Text property and a Value property. The value of the Text property is displayed in the TreeView, while the Value property is used to store any additional data about the node, such as data that is passed to the postback event that is associated with the node.
- A node can be in one of two modes: selection mode and navigation mode. By default, a node is in selection mode. To put a node into navigation mode, set the NavigateUrl property for the node to a value other than an empty string (""). To put a node into selection mode, set the NavigateUrl property for the node to an empty string ("").

- The TreeView control provides several events that you can program against. This allows you to run a custom routine whenever an event occurs. The following table lists the events that are supported by the TreeView control.

Event	Description
TreeNodeCheckChanged	Occurs when the check boxes of the TreeView control change state between posts to the server.
SelectedNodeChanged	Occurs when a node is selected in the TreeView control.
TreeNodeExpanded	Occurs when a node is expanded in the TreeView control.
TreeNodeCollapsed	Occurs when a node is collapsed in the TreeView control.
TreeNodePopulate	Occurs when a node with its PopulateOnDemand property set to <code>true</code> is expanded in the TreeView control.
TreeNodeDataBound	Occurs when a data item is bound to a node in the TreeView control.

Themes and Master Pages:

Themes

- A theme decides the look and feel of the website. It is a collection of files that define the looks of a page. It can include skin files, CSS files & images.
- We define themes in a special App_Themes folder. Inside this folder is one or more subfolders named Theme1, Theme2 etc. that define the actual themes.
- The theme property is applied late in the page's life cycle, effectively overriding any customization you may have for individual controls on your page.

How Themes Work

- There are 3 different options to apply themes to our website:
- Setting the theme at the page level: the Theme attribute is added to the page directive of the page.

```
<%@ Page Language="C#" AutoEventWireup="true" CodeFile="Default.aspx.cs" Inherits="Default" Theme="Theme1"%>
```

- 2. Setting the theme at the site level: to set the theme for the entire website you can set the theme in the web.config of the website. Open the web.config file and locate the <pages> element and add the theme attribute to it:

```
<pages theme="Theme1">
```

```
....
```

```
....
```

```
</pages>
```

- Setting the theme programmatically at runtime: here the theme is set at runtime through coding. It should be applied earlier in the page's life cycle ie. Page_PreInit event should be handled for setting the theme. The better option is to apply this to the Base page class of the site as every page in the site inherits from this class.

```
Page.Theme = Theme1;
```

Applying a Simple Theme

- Example

Create 2 themes for the page – one with red background (Theme1) and another with an image as a background (Theme2). When the user selects a particular theme from the ListBox then that theme should be applied dynamically for the page.

- Handling Theme Conflicts

To make this change, just use the StyleSheetTheme attribute instead of the Theme attribute.

Uses of Themes

- Since themes can contain CSS files, images and skins, you can change colors, fonts, positioning and images simply by applying the desired themes.
- You can have as many themes as you want and you can switch between them by setting a single attribute in the web.config file or an individual aspx page. Also you can switch between themes programmatically.
- Setting the themes programmatically, you are offering your users a quick and easy way to change the page to their likings.
- Themes allow you to improve the usability of your site by giving users with vision problems the option to select a high contrast theme with a large font size.

Simple Master Page and Content Page, Connecting Master pages and Content Pages

- Master Pages are used when user needs a consistent look and behavior over all web pages in an application. When a user needs to attach header and footer for all the web pages in an application, the master pages are used. Master pages provide a template for all other pages in an application.
- The master pages define placeholders for the content, which are overridden for the content. The result is combination of master and content page. Every master page has one or more content pages in an application.
- The advantages of the master page are as mentioned below:
 1. They provide an object model allowing users to customize the master page from the individual content pages.
 2. They allows user design the rendering of the controls in the placeholder
 3. It is centralized with common functionality of all pages to makes updates in one place
 4. Code can be applied on one set of controls and the results to the set of pages in the application

- The @Master directive is defines in the master page. The master page contains one or more <asp:ContentPlaceholder> for an individual content. The id attribute identifies the placeholder from all present in a web page. The master page has .master extension. The syntax of the master directive is as shown below:
- <%@ Master Language="C#" CodeFile="MasterPage.master.cs" Inherits="MasterPage" %>
- To create a master page, create an ASP.NET website by clicking 'File' > 'New' > 'Website'. Right click on the Project in the solution explorer and click 'Add New Item'. In the dialog box, choose the 'Master Page' and click 'Add'. The 'MasterPage.master' appears in the solution explorer.
- The content are used for holding values of the master page placeholder control. The syntax for the content page is as shown below:
- <%Page Language="C#" MasterPageFile="~/MasterPages/Master1.master" Title="Content Page %">
- The @Page directive defines as a standard page. The content pages are saved with page1.aspx extension. The MasterPageFile is the master file which is applied to the content page. The content page binds with the respective master page. The content page looks as shown below:
- <%Page Language="C#" MasterPageFile="~/MasterPages/Master1.master" Title="Content Page1 %">
 <asp:Content ID="Content1" ContentPlaceholderID="Main" runat="server">
 Main Content
 </asp:Content>
- The @Page directive page binds the content page to a specific master page. It defines a title for the page to be merged with the master page. User can create content by adding content controls and mapping them to the contentplaceholder controls on the master page.

.NET Technologies

SIUSCS46

UNIT 3

By Shivani Deopa

ADO.NET:

Introduction

- It is a module of .Net Framework which is used to establish connection between application and data sources. Data sources can be such as SQL Server and XML.
- ADO.NET consists of classes that can be used to connect, retrieve, insert and delete data.
- All the ADO.NET classes are located into System.Data.dll and integrated with XML classes located into System.Xml.dll.
- ADO.NET provides a bridge between the front end controls and the back end database.
- The ADO.NET objects encapsulate all the data access operations and the controls interact with these objects to display data, thus hiding the details of movement of data.

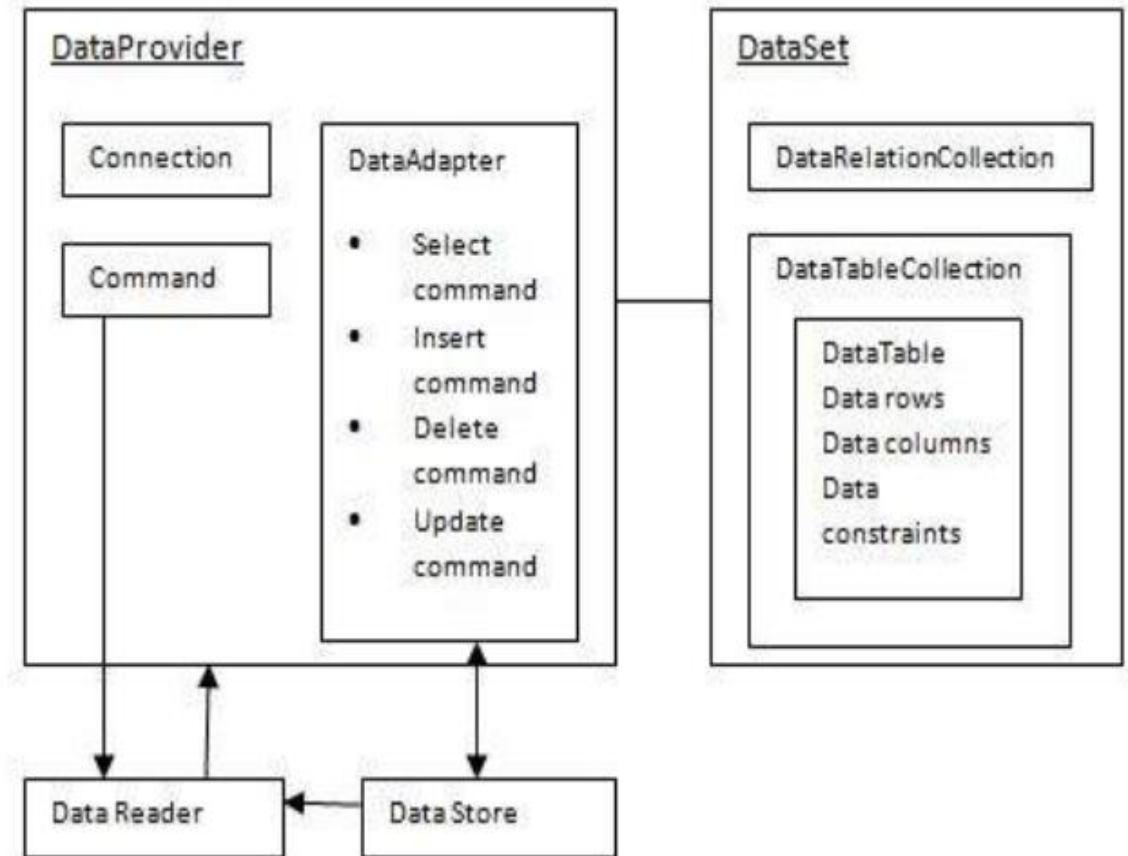
- ADO.NET has two main components that are used for accessing and manipulating data are the .NET Framework data provider and the DataSet.

- .NET Framework Data Providers

These are the components that are designed for data manipulation and fast access to data. It provides various objects such as Connection, Command, DataReader and DataAdapter that are used to perform database operations.

- The DataSet

It is used to access data independently from any data resource. DataSet contains a collection of one or more DataTable objects of data. The following diagram shows the relationship between .NET Framework data provider and DataSet.



The DataSet Class

- The dataset represents a subset of the database. It does not have a continuous connection to the database.
- To update the database a reconnection is required. The DataSet contains DataTable objects and DataRelation objects.
- The DataRelation objects represent the relationship between two tables.

- Following table shows some important properties of the DataSet class:

Properties	Description
CaseSensitive	Indicates whether string comparisons within the data tables are case-sensitive.
Container	Gets the container for the component.
DataSetName	Gets or sets the name of the current data set.
DefaultViewManager	Returns a view of data in the data set.
DesignMode	Indicates whether the component is currently in design mode.
EnforceConstraints	Indicates whether constraint rules are followed when attempting any update operation.
Events	Gets the list of event handlers that are attached to this component.
ExtendedProperties	Gets the collection of customized user information associated with the DataSet.
HasErrors	Indicates if there are any errors.
IsInitialized	Indicates whether the DataSet is initialized.
Locale	Gets or sets the locale information used to compare strings within the table.
Namespace	Gets or sets the namespace of the DataSet.
Prefix	Gets or sets an XML prefix that aliases the namespace of the DataSet.
Relations	Returns the collection of DataRelation objects.
Tables	Returns the collection of DataTable objects.

The DataTable Class

- The DataTable class represents the tables in the database. It has the following important properties; most of these properties are read only properties except the PrimaryKey property:

Properties	Description
ChildRelations	Returns the collection of child relationship.
Columns	Returns the Columns collection.
Constraints	Returns the Constraints collection.
DataSet	Returns the parent DataSet.
DefaultView	Returns a view of the table.
ParentRelations	Returns the ParentRelations collection.
PrimaryKey	Gets or sets an array of columns as the primary key for the table.
Rows	Returns the Rows collection.

- The following table shows some important methods of the DataTable class:

Methods	Description
AcceptChanges	Commits all changes since the last AcceptChanges.
Clear	Clears all data from the table.
GetChanges	Returns a copy of the DataTable with all changes made since the AcceptChanges method was called.
GetErrors	Returns an array of rows with errors.
ImportRows	Copies a new row into the table.
LoadDataRow	Finds and updates a specific row, or creates a new one, if not found any.
Merge	Merges the table with another DataTable.
NewRow	Creates a new DataRow.
RejectChanges	Rolls back all changes made since the last call to AcceptChanges.
Reset	Resets the table to its original state.
Select	Returns an array of DataRow objects.

The DataRow Class

- The DataRow object represents a row in a table. It has the following important properties and method:

Properties	Description
HasErrors	Indicates if there are any errors.
Items	Gets or sets the data stored in a specific column.
ItemArrays	Gets or sets all the values for the row.
Table	Returns the parent table.

Methods	Description
AcceptChanges	Accepts all changes made since this method was called.
BeginEdit	Begins edit operation.
CancelEdit	Cancels edit operation.
Delete	Deletes the DataRow.
EndEdit	Ends the edit operation.
GetChildRows	Gets the child rows of this row.
GetParentRow	Gets the parent row.
GetParentRows	Gets parent rows of DataRow object.
RejectChanges	Rolls back all changes made since the last call to AcceptChanges.

- The DataAdapter Object

The DataAdapter object acts as a mediator between the DataSet object and the database. This helps the Dataset to contain data from multiple databases or other data source.

- The DataReader Object

The DataReader object is an alternative to the DataSet and DataAdapter combination. This object provides a connection oriented access to the data records in the database. These objects are suitable for read-only access, such as populating a list and then breaking the connection.

- DbCommand and DbConnection Objects

The DbConnection object represents a connection to the data source. The connection could be shared among different command objects.

The DbCommand object represents the command or a stored procedure sent to the database from retrieving or manipulating data.

ADO.NET Framework Data Providers

- Data provider is used to connect to the database, execute commands and retrieve the record. It is lightweight component with better performance. It also allows us to place the data into DataSet to use it further in our application.
- The .NET Framework provides the following data providers that we can use in our application.

.NET Framework data provider	Description
.NET Framework Data Provider for SQL Server	It provides data access for Microsoft SQL Server. It requires the System.Data.SqlClient namespace.
.NET Framework Data Provider for OLE DB	It is used to connect with OLE DB. It requires the System.Data.OleDb namespace.
.NET Framework Data Provider for ODBC	It is used to connect to data sources by using ODBC. It requires the System.Data.Odbc namespace.
.NET Framework Data Provider for Oracle	It is used for Oracle data sources. It uses the System.Data.OracleClient namespace.
EntityClient Provider	It provides data access for Entity Data Model applications. It requires the System.Data.EntityClient namespace.
.NET Framework Data Provider for SQL Server Compact 4.0.	It provides data access for Microsoft SQL Server Compact 4.0. It requires the System.Data.SqlServerCe namespace.

- Following are the core object of Data Providers.

Object	Description
Connection	It is used to establish a connection to a specific data source.
Command	It is used to execute queries to perform database operations.
DataReader	It is used to read data from data source. The DbDataReader is a base class for all DataReader objects.
DataAdapter	It populates a DataSet and resolves updates with the data source. The base class for all DataAdapter objects is the DbDataAdapter class.

.NET Framework Data Provider for SQL Server

- Data provider for SQL Server is a lightweight component. It provides better performance because it directly access SQL Server without any middle connectivity layer. In early versions, it interacts with ODBC layer before connecting to the SQL Server that created performance issues.
- The .NET Framework Data Provider for SQL Server classes is located in the System.Data.SqlClient namespace. We can include this namespace in our C# application by using the following syntax- ***using System.Data.SqlClient***

Class	Description
SqlConnection	It is used to create SQL Server connection. This class cannot be inherited.
SqlCommand	It is used to execute database queries. This class cannot be inherited.
SqlDataAdapter	It represents a set of data commands and a database connection that are used to fill the DataSet. This class cannot be inherited.
SqlDataReader	It is used to read rows from a SQL Server database. This class cannot be inherited.
SqlException	This class is used to throw SQL exceptions. It throws an exception when an error is occurred. This class cannot be inherited.

Data Access - Creating a Connection

- ADO.NET SqlConnection Class- It is used to establish an open connection to the SQL Server database. It is a sealed class so that cannot be inherited.
- SqlConnection class uses SqlDataAdapter and SqlCommand classes together to increase performance when connecting to a Microsoft SQL Server database.
- Connection does not close explicitly even it goes out of scope. Therefore, you must explicitly close the connection by calling Close() method.
- Syntax

*using (SqlConnection con = new SqlConnection("data source=.;
database=Student; integrated security=SSPI"))*

• SqlConnection Constructors and Methods

Constructors	Description
SqlConnection()	It is used to initialize a new instance of the SqlConnection class.
SqlConnection(String)	It is used to initialize a new instance of the SqlConnection class and takes connection string as an argument.
SqlConnection(String, SqlConnectionCredential)	It is used to initialize a new instance of the SqlConnection class that takes two parameters. First is connection string and second is sql credentials.

Method	Description
BeginTransaction()	It is used to start a database transaction.
ChangeDatabase(String)	It is used to change the current database for an open SqlConnection.
ChangePassword(String, String)	It changes the SQL Server password for the user indicated in the connection string.
Close()	It is used to close the connection to the database.
CreateCommand()	It enlists in the specified transaction as a distributed transaction.
GetSchema()	It returns schema information for the data source of this SqlConnection.
Open()	It is used to open a database connection.
ResetStatistics()	It resets all values if statistics gathering is enabled.

Select Command

- SqlCommand Class- This class is used to store and execute SQL statement for SQL Server database. It is a sealed class so that cannot be inherited.
- This class provides the following constructors.

Constructor	Description
SqlCommand()	It is used to initialize a new instance of the SqlCommand class.
SqlCommand(String)	It is used to initialize a new instance of the SqlCommand class with a string parameter.
SqlCommand(String, SqlConnection)	It is used to initialize a new instance of the SqlCommand class. It takes two parameters, first is query string and second is connection string.
SqlCommand(String, SqlConnection, SqlTransaction)	It is used to initialize a new instance of the SqlCommand class. It takes three parameters query, connection and transaction string respectively.
SqlCommand(String, SqlConnection, SqlTransaction, SqlCommandColumnEncryptionSetting)	It Initializes a new instance of the SqlCommand class with specified command text, connection, transaction, and encryption setting.

- This class provides the following Methods.

Method	Description
BeginExecuteNonQuery()	It is used to Initiate the asynchronous execution of the SQL statement described by this SqlCommand.
Cancel()	It tries to cancel the execution of a SqlCommand.
Clone()	It creates a new SqlCommand object that is a copy of the current instance.
CreateParameter()	It creates a new instance of a SqlParameter object.
ExecuteReader()	It is used to send the CommandText to the Connection and builds a SqlDataReader.
ExecuteXmlReader()	It is used to send the CommandText to the Connection and builds an XmlReader object.
ExecuteScalar()	It executes the query and returns the first column of the first row in the result set. Additional columns or rows are ignored.
Prepare()	It is used to create a prepared version of the command by using the instance of SQL Server.
ResetCommandTimeout()	It is used to reset the CommandTimeout property to its default value.

DataReader

- ADO.NET SqlDataReader Class- This class is used to read data from SQL Server database. It reads data in forward-only stream of rows from a SQL Server database.
- It is sealed class so that cannot be inherited. It inherits DbDataReader class and implements IDisposable interface.
- SqlDataReader Properties

Property	Description
Connection	It is used to get the SqlConnection associated with the SqlDataReader.
Depth	It is used to get a value that indicates the depth of nesting for the current row.
FieldCount	It is used to get the number of columns in the current row.
HasRows	It is used to get a value that indicates whether the SqlDataReader contains one or more rows.
IsClosed	It is used to retrieve a boolean value that indicates whether the specified SqlDataReader instance has been closed.
Item[String]	It is used to get the value of the specified column in its native format given the column name.
Item[Int32]	It is used to get the value of the specified column in its native format given the column ordinal.
RecordsAffected	It is used to get the number of rows changed, inserted or deleted by execution of the Transact-SQL statement.
VisibleFieldCount	It is used to get the number of fields in the SqlDataReader that are not hidden.

- SqlDataReader Methods

Method	Description
Close()	It is used to closes the SqlDataReader object.
GetBoolean(Int32)	It is used to get the value of the specified column as a Boolean.
GetByte(Int32)	It is used to get the value of the specified column as a byte.
GetChar(Int32)	It is used to get the value of the specified column as a single character.
GetDateTime(Int32)	It is used to get the value of the specified column as a DateTime object.
GetDecimal(Int32)	It is used to get the value of the specified column as a Decimal object.
GetDouble(Int32)	It is used to get the value of the specified column as a double-precision floating point number.
GetFloat(Int32)	It is used to get the value of the specified column as a single-precision floating point number.
GetName(Int32)	It is used to get the name of the specified column.
GetSchemaTable()	It is used to get a DataTable that describes the column metadata of the SqlDataReader.
GetValue(Int32)	It is used to get the value of the specified column in its native format.
GetValues(Object[])	It is used to populate an array of objects with the column values of the current row.
NextResult()	It is used to get the next result, when reading the results of SQL statements.
Read()	It is used to read record from the SQL Server database.

DataAdapter

- ADO.NET DataAdapter- The DataAdapter works as a bridge between a DataSet and a data source to retrieve data.
- DataAdapter is a class that represents a set of SQL commands and a database connection. It can be used to fill the DataSet and update the data source.
- DataAdapter Constructors

Constructors	Description
<code>DataAdapter()</code>	It is used to initialize a new instance of a DataAdapter class.
<code>DataAdapter(DataAdapter)</code>	It is used to initializes a new instance of a DataAdapter class from an existing object of the same type.

- DataAdapter Methods

Method	Description
CloneInternals()	It is used to create a copy of this instance of DataAdapter.
Dispose(Boolean)	It is used to release the unmanaged resources used by the DataAdapter.
Fill(DataSet)	It is used to add rows in the DataSet to match those in the data source.
FillSchema(DataSet, SchemaType, String, IDataReader)	It is used to add a DataTable to the specified DataSet.
GetFillParameters()	It is used to get the parameters set by the user when executing an SQL SELECT statement.
ResetFillLoadOption()	It is used to reset FillLoadOption to its default state.
ShouldSerializeAcceptChangesDuringFill()	It determines whether the AcceptChangesDuringFill property should be persisted or not.
ShouldSerializeFillLoadOption()	It determines whether the FillLoadOption property should be persisted or not.
ShouldSerializeTableMappings()	It determines whether one or more DataTableMapping objects exist or not.
Update(DataSet)	It is used to call the respective INSERT, UPDATE, or DELETE statements.

- Example:

```
SqlConnection con = null;
```

```
try
```

```
{
```

```
    // Creating Connection
```

```
    con = new SqlConnection("data source=.; database=Student; integrated security=SSPI");
```

```
    // writing sql query
```

```
    SqlCommand retrieve = new SqlCommand("Select * from student", con);
```

```
    // Opening Connection
```

```
    con.Open();
```

```
    // Executing the SQL query
```

```
    SqlDataReader sdr = retrieve.ExecuteReader();
```

```
    // Iterating Data
```

```
    while (sdr.Read())
```

```
    {        Console.WriteLine(sdr["id"] + " " + sdr["name"] + " " + sdr["email"]); // Displaying Record    }
```

```
}
```

```
catch (Exception e) {    Console.WriteLine("OOPs, something went wrong." + e);    }
```

```
// Closing the connection
```

```
finally {    con.Close();    }
```

Data Controls:

GridView

- The GridView control is used to display the values of a data source in a table. Each column represents a field, while each row represents a record.
- The GridView control supports the following features:
 1. Binding to data source controls, such as SqlDataSource.
 2. Built-in sort capabilities.
 3. Built-in update and delete capabilities.
 4. Built-in paging capabilities.
 5. Built-in row selection capabilities.
 6. Programmatic access to the GridView object model to dynamically set properties, handle events, and so on.
 7. Multiple key fields.
 8. Multiple data fields for the hyperlink columns.
 9. Customizable appearance through themes and styles.
 10. To learn about the other data-bound controls that are available in ASP.NET, see Data-Bound Web Server Controls.

Column Fields

- Each column in the GridView control is represented by a DataControlField object. By default, the AutoGenerateColumns property is set to true, which creates an AutoGeneratedField object for each field in the data source.
- Each field is then rendered as a column in the GridView control in the order that each field appears in the data source.
- You can also manually control which column fields appear in the GridView control by setting the AutoGenerateColumns property to false and then defining your own column field collection.
- Different column field types determine the behavior of the columns in the control. The following table lists the different column field types that can be used.

Column field type	Description
BoundField	Displays the value of a field in a data source. This is the default column type of the GridView control.
ButtonField	Displays a command button for each item in the GridView control. This enables you to create a column of custom button controls, such as the Add or the Remove button.
CheckBoxField	Displays a check box for each item in the GridView control. This column field type is commonly used to display fields with a Boolean value.
CommandField	Displays predefined command buttons to perform select, edit, or delete operations.
HyperLinkField	Displays the value of a field in a data source as a hyperlink. This column field type enables you to bind a second field to the hyperlink's URL.
ImageField	Displays an image for each item in the GridView control.
TemplateField	Displays user-defined content for each item in the GridView control according to a specified template. This column field type enables you to create a custom column field.


```
<asp:gridview id="CustomersGridView" datasourceid="CustomersSource"
autogeneratecolumns="False" emptydatatext="No data available."
allowpaging="True" runat="server" DataKeyNames="CustomerID">
<Columns>
<asp:BoundField DataField="CustomerID" HeaderText="CustomerID"
InsertVisible="False" ReadOnly="True" SortExpression="CustomerID" />
<asp:BoundField DataField="CompanyName" HeaderText="CompanyName"
SortExpression="CompanyName" />
<asp:BoundField DataField="FirstName" HeaderText="FirstName"
SortExpression="FirstName" />
<asp:BoundField DataField="LastName" HeaderText="LastName"
SortExpression="LastName" />
</Columns>
</asp:gridview>
```

DetailsView

- The DetailsView control is used to display a single record from a data source in a table, where each field of the record is displayed in a row of the table.
- It can be used in combination with a GridView control for master-detail scenarios. The DetailsView control supports the following features:
 1. Binding to data source controls, such as SqlDataSource.
 2. Built-in inserting capabilities.
 3. Built-in updating and deleting capabilities.
 4. Built-in paging capabilities.
 5. Programmatic access to the DetailsView object model to dynamically set properties, handle events, and so on.
 6. Customizable appearance through themes and styles.

Row Fields

- Each data row in the DetailsView control is created by declaring a field control. Different row field types determine the behavior of the rows in the control.
- Field controls derive from DataControlField. The following table lists the different row field types that can be used.

Column field type	Description
BoundField	Displays the value of a field in a data source as text.
ButtonField	Displays a command button in the DetailsView control. This allows you to display a row with a custom button control, such as an Add or a Remove button.
CheckBoxField	Displays a check box in the DetailsView control. This row field type is commonly used to display fields with a Boolean value.
CommandField	Displays built-in command buttons to perform edit, insert, or delete operations in the DetailsView control.
HyperLinkField	Displays the value of a field in a data source as a hyperlink. This row field type allows you to bind a second field to the hyperlink's URL.
ImageField	Displays an image in the DetailsView control.
TemplateField	Displays user-defined content for a row in the DetailsView control according to a specified template. This row field type allows you to create a custom row field.

```
<asp:DetailsView ID="DetailsView" runat="server" DataSourceID="Details"
AutoGenerateRows="false" DataKeyNames="CustomerID" > <HeaderStyle
BackColor="Navy" ForeColor="White" />
<Fields>
<asp:BoundField DataField="CustomerID" HeaderText="CustomerID"
ReadOnly="True" />
<asp:BoundField DataField="ContactName" HeaderText="ContactName" />
<asp:BoundField DataField="ContactTitle" HeaderText="ContactTitle" />
<asp:BoundField DataField="CompanyName" HeaderText="CompanyName" />
<asp:BoundField DataField="City" HeaderText="City" />
<asp:BoundField DataField="Region" HeaderText="Region" />
<asp:BoundField DataField="PostalCode" HeaderText="PostalCode" />
<asp:BoundField DataField="Country" HeaderText="Country" />
</Fields>
</asp:DetailsView>
```

FormView

- The FormView control is used to display a single record from a data source. It is similar to the DetailsView control, except it displays user-defined templates instead of row fields.
- Creating your own templates gives you greater flexibility in controlling how the data is displayed.
- The FormView control supports the following features:
 1. Binding to data source controls, such as SqlDataSource and ObjectDataSource.
 2. Built-in inserting capabilities.
 3. Built-in updating and deleting capabilities.
 4. Built-in paging capabilities.
 5. Programmatic access to the FormView object model to dynamically set properties, handle events, and so on.
 6. Customizable appearance through user-defined templates, themes, and styles.

Templates

- For the FormView control to display content, you need to create templates for the different parts of the control.
- Most templates are optional; however, you must create a template for the mode in which the control is configured.
- For example, a FormView control that supports inserting records must have an insert item template defined. The following table lists the different templates that you can create.

Template type	Description
EditItemTemplate	Defines the content for the data row when the FormView control is in edit mode. This template usually contains input controls and command buttons with which the user can edit an existing record.
EmptyDataTemplate	Defines the content for the empty data row displayed when the FormView control is bound to a data source that does not contain any records. This template usually contains content to alert the user that the data source does not contain any records.
FooterTemplate	Defines the content for the footer row. This template usually contains any additional content you would like to display in the footer row. Note: As an alternative, you can simply specify text to display in the footer row by setting the FooterText property.
HeaderTemplate	Defines the content for the header row. This template usually contains any additional content you would like to display in the header row. Note: As an alternative, you can simply specify text to display in the header row by setting the HeaderText property.
ItemTemplate	Defines the content for the data row when the FormView control is in read-only mode. This template usually contains content to display the values of an existing record.
InsertItemTemplate	Defines the content for the data row when the FormView control is in insert mode. This template usually contains input controls and command buttons with which the user can add a new record.
PagerTemplate	Defines the content for the pager row displayed when the paging feature is enabled (when the AllowPaging property is set to <code>true</code>). This template usually contains controls with which the user can navigate to another record. Note: The FormView control has a built-in pager row user interface (UI). You need to create a pager template only if you want to create your own custom pager row.

```
<asp:formview id="studentviewform" datasourceid="student" allowpaging="true"
datakeynames="" runat="server">
<itemtemplate>
<table>
<tr>
<td>
<h3><%# Eval("name") %></h3>
<h4><%# Eval("email") %></h4>
</td>
</tr>
</table>
</itemtemplate>
<pagersettings position="Bottom" mode="NextPrevious"/>
</asp:formview>
```

Data Binding:

Introduction

- Every ASP.NET web form control inherits the DataBind method from its parent Control class, which gives it an inherent capability to bind data to at least one of its properties. This is known as simple data binding or inline data binding.
- Simple data binding involves attaching any collection (item collection) which implements the IEnumerable interface, or the DataSet and DataTable classes to the DataSource property of the control.
- On the other hand, some controls can bind records, lists, or columns of data into their structure through a DataSource control.
- These controls derive from the BaseDataBoundControl class. This is called declarative data binding.
- The data source controls help the data-bound controls implement functionalities such as, sorting, paging, and editing data collections.

- The BaseDataBoundControl is an abstract class, which is inherited by two more abstract classes:
 1. DataBoundControl
 2. HierarchicalDataBoundControl
- The abstract class DataBoundControl is again inherited by two more abstract classes:
 1. ListControl
 2. CompositeDataBoundControl
- The controls capable of simple data binding are derived from the ListControl abstract class and these controls are:
 1. BulletedList
 2. CheckBoxLayout
 3. DropDownList
 4. ListBox
 5. RadioButtonList
- The controls capable of declarative data binding (a more complex data binding) are derived from the abstract class CompositeDataBoundControl. These controls are:
 1. DetailsView
 2. FormView
 3. GridView
 4. RecordList

Single-Value Data Binding

- Single-value data binding is really just a different approach to dynamic text. To use it, you add special data binding expressions into your .aspx files. These expressions have the following format:

`<%# expression_goes_here %>`

- This may look like a script block, but it isn't. If you try to write any code inside this tag, you will receive an error. The only thing you can add is a valid data binding expression.
- For example, if you have a public or protected variable named Country in your page, you could write the following in .aspx page:

`<%# Country %>`

- When you call the DataBind() method for the page, this text will be replaced with the value for Country (for example, Spain).

- Example:

```
<asp:Label ID="Label1" runat="server" Font-Size="Large">
```

```
    User ID: <%# UserID %><br />
```

```
    UserName: <%# UserName %><br />
```

```
    City: <%# City %>
```

```
</asp:Label>
```

```
public partial class SingleValueDataBinding : System.Web.UI.Page
```

```
{  
    public int UserID;  
    public string UserName;  
    public string City;
```

```
    protected void Page_Load(object sender, EventArgs e)
```

```
{  
    UserID = 1;  
    UserName = "Jones";  
    City = "Rome";  
    this.DataBind();
```

```
}
```

```
}
```

Repeated-Value Data Binding

- Although using simple data binding is optional, repeated-value binding is so useful that almost every ASP.NET application will want to use it somewhere.
- Repeated-value data binding works with the ASP.NET list controls (and the rich data controls).
- To use repeated-value binding, you link one of these controls to a data source (such as a field in a data table).
- When you call `DataBind()`, the control automatically creates a full list using all the corresponding values.
- This saves you from writing code that loops through the array or data table and manually adds elements to a control.

- Example:

```
protected void Page_Load(object sender, EventArgs e)
{
    //Create an ArrayList of values to bind to
    ArrayList MyArrayList = new ArrayList();
    MyArrayList.Add("Turbo C#");
    MyArrayList.Add("C# Programming 101");
    MyArrayList.Add("DOS Basic Training");
    MyArrayList.Add("C# Patterns with ASP.NET");
    MyArrayList.Add("ADO.NET in Action");
    MyArrayList.Add("Intermediate ASP.NET");

    //Bind the controls to the data source MyArrayList
    MyDropDownList.DataSource = MyArrayList;
    MyRadioButtonList.DataSource = MyArrayList;
    MyDataList.DataSource = MyArrayList;

    //Bind all controls on the page
    Page.DataBind();
}
```

Data Source Controls – SqlDataSource

- The SqlDataSource control represents a connection to a relational database such as SQL Server or Oracle database, or data accessible through OLEDB or Open Database Connectivity (ODBC).
- Connection to data is made through two important properties ConnectionString and ProviderName.
- The following code snippet provides the basic syntax of the control:

```
<asp:SqlDataSource runat="server" ID="MySqlSource"  
    ProviderName='<%%$ ConnectionStrings:LocalNWind.ProviderName %>'  
    ConnectionString='<%%$ ConnectionStrings:LocalNWind %>'  
    SelectionCommand= "SELECT * FROM EMPLOYEES" />
```

```
<asp:GridView ID="GridView1" runat="server" DataSourceID="MySqlSource" />
```

- Configuring various data operations on the underlying data depends upon the various properties (property groups) of the data source control.

- The following table provides the related sets of properties of the SqlDataSource control, which provides the programming interface of the control:

Property Group	Description
DeleteCommand, DeleteParameters, DeleteCommandType	Gets or sets the SQL statement, parameters, and type for deleting rows in the underlying data.
FilterExpression, FilterParameters	Gets or sets the data filtering string and parameters.
InsertCommand, InsertParameters, InsertCommandType	Gets or sets the SQL statement, parameters, and type for inserting rows in the underlying database.
SelectCommand, SelectParameters, SelectCommandType	Gets or sets the SQL statement, parameters, and type for retrieving rows from the underlying database.
SortParameterName	Gets or sets the name of an input parameter that the command's stored procedure will use to sort data.
UpdateCommand, UpdateParameters, UpdateCommandType	Gets or sets the SQL statement, parameters, and type for updating rows in the underlying data store.

Working with XML:

XML Introduction

- XML is short for eXtensible Markup Language. It is a very widely used format for exchanging data, mainly because it's easy readable for both humans and machines.
- If you have ever written a website in HTML, XML will look very familiar to you, as it's basically a stricter version of HTML. XML is made up of tags, attributes and values and looks something like this:

```
<users>
```

```
<user name="John Doe" age="42" />
```

```
<user name="Jane Doe" age="39" />
```

```
</users>
```

- We use System.Xml namespace, to deal with pretty much any aspect of XML in C#/ASP .Net.

Reading XML with the XmlReader class

- There are mainly two methods for reading XML with C#: The XmlDocument class and the XmlReader class.
- XmlDocument reads the entire XML content into memory and then lets you navigate back and forward in it as you please, or even query the document using the XPath technology.
- The XmlReader is a faster and less memory-consuming alternative. It lets you run through the XML content one element at a time, while allowing you to look at the value, and then moves on to the next element.
- By doing so, it obviously consumes very little memory because it only holds the current element, and because you have to manually check the values, you will only get the relevant elements, making it very fast.

```

XmlReader xmlReader = XmlReader.Create("EuropeanCentralBank.xml");
    while(xmlReader.Read())
    {
        if((xmlReader.NodeType == XmlNodeType.Element) && (xmlReader.Name == "Cube"))
        {
            if(xmlReader.HasAttributes)
                Console.WriteLine(xmlReader.GetAttribute("currency") + ": " + xmlReader.GetAttribute("rate"));
        }
    }
    Console.ReadKey();


```

- We start off by creating the XmlReader instance, using the static Create() method. It has several overloads, but the simplest of them just takes a URL pointing to the file.
- In a while loop, we call the Read() method on the XmlReader instance. It advances the reader to the next element and then returns true as long as there is more to read.
- Inside the loop, we can now use one of the many properties and methods on the XmlReader to access data from the current element.
- In our case, we check the NodeType to see if we have an Element (the tag part), and if the name is "Cube". As you can see in the XML document we used, each currency rate is within an element called Cube, so this is of course what we're looking for - the rest is ignored.
- Once we have a Cube element, we do a formal check to see if it has attributes (which it should) and then we use the GetAttribute() method to read the two attribute values "currency" and "rate".
- We print them out to the Console and then we move on to the next element. The result should be a complete list of currencies and their current exchange rate.

Reading XML with the XmlDocument class

- The XmlDocument is more memory consuming and possibly a bit slower than the XmlReader approach. However, for many purposes, the XmlDocument can be easier to work with and often require less code.
- Once the XML content has been read, you can read the data in a hierarchical way, just like the XML structure, with a root element which can have child elements, which can have child elements, and so on.
- Previously we parsed XML data from the European Central Bank which could tell us about the current currency exchange rates, and we will do the same now, but using the XmlDocument class instead.

```
<gesmes:Envelope>  
  [other child nodes]  
  <Cube>  
    <Cube time="2011-04-12">  
      <Cube currency="USD" rate="1.4470"/>  
      <Cube currency="JPY" rate="121.87"/>  
    ...
```

- The gesmes:Envelope is our root element, which we can access using the DocumentElement property.
- We will then be able to access children of this node by using the ChildNodes collection property. In our example, we want the child nodes three levels below the root/document element.

```
XmlDocument xmlDoc = new XmlDocument();  
xmlDoc.Load("EuropeanCentralBank.xml");  
foreach(XmlNode xmlNode in  
    xmlDoc.DocumentElement.ChildNodes[2].ChildNodes[0].ChildNodes)  
    Console.WriteLine(xmlNode.Attributes["currency"].Value + ": " +  
        xmlNode.Attributes["rate"].Value);  
Console.ReadKey();
```

- As you can see, we access the Cube nodes by going down the ChildNodes hierarchy.
- From the DocumentElement (the root element), we ask for the third child node (zero-index based), then we ask for the first child node of that, and then we ask for the entire collection of child nodes.
- Obviously this is only possible because we know the structure of the XML document, and it's definitely not very flexible, pretty or easy to change later on.
- However, the way you navigate an XML document very much depends on the XML source and the data you need.
- For this example, the above will work just fine and even with a very limited amount of code, but for other purposes, you may want to use a bit more code to increase the readability.

Working with the XmlNode class

- The XML is basically parsed into an XmlNode which is the root element and then you can access the child elements using the ChildNodes property.
- However, the XmlNode class gives you access to a lot of other information as well, for instance the name of the tag, the attributes, the inner text and the XML itself.
- This chapter is a brief description of some of the more interesting aspects of the XmlNode class, which is important to know about because the XmlNode class is such a key concept when parsing XML with the XmlDocument class.
- In the following examples we will use the DocumentElement a lot, and while it is in fact of the type XmlElement, XmlElement does inherit from XmlNode, so it's essentially the same.
- The Name property will simply give you the name of the node. For instance, the following example will output the text "user":

```
XmlDocument xmlDoc = new XmlDocument();  
xmlDoc.LoadXml("<user name=\"John Doe\">A user node</user>");  
Console.WriteLine(xmlDoc.DocumentElement.Name);  
Console.ReadKey();
```

Writing XML with the XmlWriter class

- Since XML is simply text, you could just start writing out XML tags to a file and give it an xml extension, but it's more easy and safer to let the .NET framework handle it, and just like with reading XML, there are at least two different options: The XmlWriter approach and the XmlDocument approach.
- The difference between the two is once again mostly related to memory consumption - XmlWriter uses less memory than XmlDocument, which is only an issue if you write very big files though.
- Another important difference is that when using XmlDocument, you can read an existing file, manipulate it and then write back the changes.
- With XmlWriter, you will have to write the entire document from scratch each time.
- This is not necessarily a problem though, so as always, it really comes down to your situation as well as your personal preferences

- Here's an example of writing XML using the XmlWriter class:

```
XmlWriter xmlWriter = XmlWriter.Create("test.xml");  
xmlWriter.WriteStartDocument();  
xmlWriter.WriteStartElement("users");  
xmlWriter.WriteStartElement("user");  
xmlWriter.WriteAttributeString("age", "42");  
xmlWriter.WriteString("John Doe");  
xmlWriter.WriteEndElement();  
xmlWriter.WriteStartElement("user");  
xmlWriter.WriteAttributeString("age", "39");  
xmlWriter.WriteString("Jane Doe");  
xmlWriter.WriteEndDocument();  
xmlWriter.Close();
```

- The above code will create the following XML:

```
<users>  
  <user age="42">John Doe</user>  
  <user age="39">Jane Doe</user>  
</users>
```

- So, we start by creating an instance of the XmlWriter class. It takes at least one parameter, in this case the path to where the XML should be written, but comes in many variations for various purposes.
- The first thing we should do is to call the WriteStartDocument() method. After that, we write a start element called "users". The XmlWriter will translate this into <users>.
- Before closing it, we write another start element, "user", which will then become a child of "users". We then proceed to adding an attribute (age) to the element, using the WriteAttributeString() method, and then we write the inner text of the element, by calling the WriteString() method.
- We then make sure to close the first "user" element with a call to the WriteEndElement() method.
- This process is repeated to add another user, except that we don't call WriteEndElement() like we did before.
- In fact, it should be called twice, since we have an open "users" element too, but the XmlWriter will do that for us when we call the WriteEndDocument() method.
- To get the XmlWriter to write our data to the disk, we call the Close() method. You can now open the file, "test.xml", in the same directory where the EXE file of your project is, usually in the bin\debug directory.
- And that's really all it takes to write a simple XML file.

Caching:

When to Use Caching

- Caching is a technique of storing frequently used data/information in memory, so that, when the same data/information is needed next time, it could be directly retrieved from the memory instead of being generated by the application.
- Caching is extremely important for performance boosting in ASP.NET, as the pages and controls are dynamically generated here. It is especially important for data related transactions, as these are expensive in terms of response time.
- Caching places frequently used data in quickly accessed media such as the random access memory of the computer. The ASP.NET runtime includes a key-value map of CLR objects called cache.
- This resides with the application and is available via the HttpContext and System.Web.UI.Page.
- In some respect, caching is similar to storing the state objects. However, the storing information in state objects is deterministic, i.e., you can count on the data being stored there, and caching of data is nondeterministic.
- The data will not be available in the following cases:
 1. If its lifetime expires,
 2. If the application releases its memory,
 3. If caching does not take place for some reason.
- You can access items in the cache using an indexer and may control the lifetime of objects in the cache and set up links between the cached objects and their physical sources.

Caching in ASP.Net

- ASP.NET provides the following different types of caching:
 1. Output Caching : Output cache stores a copy of the finally rendered HTML pages or part of pages sent to the client. When the next client requests for this page, instead of regenerating the page, a cached copy of the page is sent, thus saving time.
 2. Data Caching : Data caching means caching data from a data source. As long as the cache is not expired, a request for the data will be fulfilled from the cache. When the cache is expired, fresh data is obtained by the data source and the cache is refilled.
 3. Object Caching : Object caching is caching the objects on a page, such as data-bound controls. The cached data is stored in server memory.
 4. Class Caching : Web pages or web services are compiled into a page class in the assembly, when run for the first time. Then the assembly is cached in the server. Next time when a request is made for the page or service, the cached assembly is referred to. When the source code is changed, the CLR recompiles the assembly.
 5. Configuration Caching : Application wide configuration information is stored in a configuration file. Configuration caching stores the configuration information in the server memory.

Output Caching

- Rendering a page may involve some complex processes such as, database access, rendering complex controls etc.
- Output caching allows bypassing the round trips to server by caching data in memory. Even the whole page could be cached.
- The OutputCache directive is responsible of output caching. It enables output caching and provides certain control over its behaviour.
- Syntax for OutputCache directive:

<%@ OutputCache Duration="15" VaryByParam="None" %>

- The OutputCache directive has the following attributes, which helps in controlling the behaviour of the output cache:

Attribute	Values	Description
DiskCacheable	true/false	Specifies that output could be written to a disk based cache.
NoStore	true/false	Specifies that the "no store" cache control header is sent or not.
CacheProfile	String name	Name of a cache profile as to be stored in web.config.
VaryByParam	None * Param- name	Semicolon delimited list of string specifies query string values in a GET request or variable in a POST request.
VaryByHeader	* Header names	Semicolon delimited list of strings specifies headers that might be submitted by a client.
VaryByCustom	Browser Custom string	Tells ASP.NET to vary the output cache by browser name and version or by a custom string.
Location	Any Client Downstream Server None	Any: page may be cached anywhere. Client: cached content remains at browser. Downstream: cached content stored in downstream and server both. Server: cached content saved only on server. None: disables caching.
Duration	Number	Number of seconds the page or control is cached.

Data Caching

- The main aspect of data caching is caching the data source controls. We have already discussed that the data source controls represent data in a data source, like a database or an XML file.
- These controls derive from the abstract class `DataSourceControl` and have the following inherited properties for implementing caching:
- `CacheDuration` - It sets the number of seconds for which the data source will cache data.
- `CacheExpirationPolicy` - It defines the cache behavior when the data in cache has expired.
- `CacheKeyDependency` - It identifies a key for the controls that auto-expires the content of its cache when removed.
- `EnableCaching` - It specifies whether or not to cache the data.

- Example:

```
<asp:SqlDataSource ID = "SqlDataSource1" runat = "server"  
    ConnectionString = "<%%$ ConnectionStrings: connectionstringID %>"  
    ProviderName = "<%%$ ConnectionStrings: SQLProviderName %>"  
    SelectCommand = "SELECT * FROM [dbTablename]"  
    EnableCaching = "true" CacheDuration = "60">  
</asp:SqlDataSource>
```

- The EnableCaching attribute of the data source control is 'true' and the CacheDuration attribute is set as '60'. It will implement caching and the cache will expire every 60 seconds.
- The timestamp changes with every refresh, but if you change the data in the table within these 60 seconds, it is not shown before the cache expires.

LINQ:

Understanding LINQ

- Most applications are data-centric, however most of the data repositories are relational databases. Over the years, designers and developers have designed applications based on object models.
- The objects are responsible for connecting to the data access components - called the Data Access Layer (DAL). Here we have three points to consider:
 1. All the data needed in an application are not stored in the same source. The source could be a relation database, some business object, XML file, or a web service.
 2. Accessing in-memory object is simpler and less expensive than accessing data from a database or XML file.
 3. The data accessed are not used directly, but needs to be sorted, ordered, grouped, altered etc.
- Hence if there is one tool that makes all kind of data access easy that allows joining data from such disparate data sources and perform standard data processing operations, in few lines of codes, it would be of great help.
- LINQ or Language-Integrated Query is such a tool. LINQ is set of extensions to the .Net Framework 3.5 and its managed languages that set the query as an object. It defines a common syntax and a programming model to query different types of data using a common language.

LINQ Basics

- The relational operators like Select, Project, Join, Group, Partition, Set operations etc., are implemented in LINQ and the C# and VB compilers in the .Net framework, which support the LINQ syntax makes it possible to work with a configured data store without resorting to ADO.NET.
- For example, querying the Customers table in the Northwind database, using LINQ query in C#, the code would be:

```
var data = from c in dataContext.Customers  
where c.Country == "Spain"  
select c;
```

- Where:

The 'from' keyword logically loops through the contents of the collection.

The expression with the 'where' keyword is evaluated for each object in the collection.

The 'select' statement selects the evaluated object to add to the list being returned.

The 'var' keyword is for variable declaration.

- Since the exact type of the returned object is not known, it indicates that the information will be inferred dynamically.

LINQ Operators

- Example: Suppose we have List<Books> and IEnumerable<Salesdetails>
- Select query on Book in LINQ will be

```
var booktitles =  
from b in books  
select b.Title;
```

- Is equivalent to the following SQL query:

```
SELECT Title from Books
```

- The Join clause- The 'join clause' in SQL is used for joining two data tables and displays a data set containing columns from both the tables. LINQ is also capable of that.

```
var booktitles = from b in books join s in sales on b.ID equals s.ID  
select new { Name = b.Title, Pages = s.pages };
```

- The Where clause- The 'where clause' allows adding some conditional filters to the query. For example, if you want to see the books, where the number of pages are more than 500, write the following code :

```
var booktitles = from b in books join s in sales on b.ID equals s.ID  
where s.pages > 500 select new { Name = b.Title, Pages = s.pages };
```

- Orderby and Orderbydescending Clauses- These clauses allow sorting the query results. To query the titles, number of pages and price of the book, sorted by the price, write the following code:

```
var booktitles = from b in books join s in sales on b.ID equals s.ID  
orderby b.Price select new { Name = b.Title, Pages = s.pages, Price = b.Price};
```

- The Let clause- The let clause allows defining a variable and assigning it a value calculated from the data values. For example, to calculate the total sale from the above two sales, you need to calculate:

*TotalSale = Price of the Book * Sales*

- To achieve this, use the following code:
- The let clause allows defining a variable and assigning it a value calculated from the data values. For example, to calculate the total sale from the above two sales, you need to calculate:

```
var booktitles = from b in book join s in sales on b.ID equals s.ID  
let totalprofit = (b.Price * s.sales)  
select new { Name = b.Title, TotalSale = totalprofit};
```

ASP.NET AJAX:

ScriptManager

- AJAX stands for Asynchronous JavaScript and XML. This is a cross platform technology which speeds up response time. The AJAX server controls add script to the page which is executed and processed by the browser.
- However like other ASP.NET server controls, these AJAX server controls also can have methods and event handlers associated with them, which are processed on the server side.
- The control toolbox in the Visual Studio IDE contains a group of controls called the 'AJAX Extensions'

The ScriptManager Control

- The ScriptManager control is the most important control and must be present on the page for other controls to work.
- It has the basic syntax:

```
<asp:ScriptManager ID="ScriptManager1" runat="server">  
</asp:ScriptManager>
```

- If you create an 'Ajax Enabled site' or add an 'AJAX Web Form' from the 'Add Item' dialog box, the web form automatically contains the script manager control. The ScriptManager control takes care of the client-side script for all the server side controls.

Partial Refreshes

- The UpdatePanel control is a container control and derives from the Control class. It acts as a container for the child controls within it and does not have its own interface.
- When a control inside it triggers a post back, the UpdatePanel intervenes to initiate the post asynchronously and update just that portion of the page.
- For example, if a button control is inside the update panel and it is clicked, only the controls within the update panel will be affected, the controls on the other parts of the page will not be affected.
- This is called the partial post back or the asynchronous post back.
- The <asp:UpdatePanel> tag has two childtags - the ContentTemplate and the Triggers tags.
- The ContentTemplate tag is required, since it holds the content of the panel. The content can be anything that you would normally put on your page, from literal text to web controls.
- The Triggers tag allows you to define certain triggers which will make the panel update it's content.

- Properties of the UpdatePanel Control

Properties	Description
ChildrenAsTriggers	This property indicates whether the post backs are coming from the child controls, which cause the update panel to refresh.
ContentTemplate	It is the content template and defines what appears in the update panel when it is rendered.
ContentTemplateContainer	Retrieves the dynamically created template container object and used for adding child controls programmatically.
IsInPartialRendering	Indicates whether the panel is being updated as part of the partial post back.
RenderMode	Shows the render modes. The available modes are Block and Inline.
UpdateMode	Gets or sets the rendering mode by determining some conditions.
Triggers	Defines the collection trigger objects each corresponding to an event causing the panel to refresh automatically.

- Methods of the UpdatePanel Control

Methods	Description
CreateContentTemplateContainer	Creates a Control object that acts as a container for child controls that define the UpdatePanel control's content.
CreateControlCollection	Returns the collection of all controls that are contained in the UpdatePanel control.
Initialize	Initializes the UpdatePanel control trigger collection if partial-page rendering is enabled.
Update	Causes an update of the content of an UpdatePanel control.

Progress Notification

- One of the problems with Ajax, is the fact that since it's asynchronous and in the background, the browser will not show you any status.
- With fast servers and fast methods, this is not a big problem, but whenever you have a method which takes up a bit of time, the user is very likely to get impatient.
- The UpdateProgress control provides a sort of feedback on the browser while one or more update panel controls are being updated.
- For example, while a user logs in or waits for server response while performing some database oriented job.
- It provides a visual acknowledgement like "Loading page...", indicating the work is in progress.
- You can even have multiple UpdateProgress controls on the page, and by using the AssociatedUpdatePanelID property, you can make sure that the UpdateProgress is only shown when a certain UpdatePanel is updated.
- The DynamicLayout property is nice to know as well. It tells whether or not the page should reserve space for your progress control.
- If it's set to true, which is the default, the space is dynamic, hence it's not reserved, but taken when the control is shown. If you wish to reserve the space, set this property to false.

- Properties and Methods of the UpdateProgress Control

Properties	Description
AssociatedUpdatePanelID	Gets and sets the ID of the update panel with which this control is associated.
Attributes	Gets or sets the cascading style sheet (CSS) attributes of the UpdateProgress control.
DisplayAfter	Gets and sets the time in milliseconds after which the progress template is displayed. The default is 500.
DynamicLayout	Indicates whether the progress template is dynamically rendered.
ProgressTemplate	Indicates the template displayed during an asynchronous post back which takes more time than the DisplayAfter time.

Methods	Description
GetScriptDescriptors	Returns a list of components, behaviors, and client controls that are required for the UpdateProgress control's client functionality.
GetScriptReferences	Returns a list of client script library dependencies for the UpdateProgress control.

Timed Refreshes

- Timer controls allow you to do postbacks at certain intervals. If used together with UpdatePanels, which is the most common approach, it allows for timed partial updates of your page, but it can be used for posting back the entire page as well.

(1) Setting the Triggers property of the UpdatePanel control:

<Triggers>

<asp:AsyncPostBackTrigger ControlID="btnpanel2" EventName="Click" />

</Triggers>

(2) Placing a timer control directly inside the UpdatePanel to act as a child control trigger. A single timer can be the trigger for multiple UpdatePanels.

<asp:UpdatePanel ID="UpdatePanel1" runat="server" UpdateMode="Always">

<ContentTemplate>

<asp:Timer ID="Timer1" runat="server" Interval="1000"></asp:Timer>

<asp:Label ID="Label1" runat="server" Height="101px" style="width:304px" ></asp:Label>

</ContentTemplate>

</asp:UpdatePanel>